DICE: Decentralized Incentive-driven Federated Learning for Efficient & Scalable Customization of Pretrained Models in Web3

Hong Lin
The State Key Laboratory of Blockchain and Data Security, Zhejiang University
Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain&Data Security
honglin@zju.edu.cn

Zhongle Xie
The State Key Laboratory of Blockchain and Data Security, Zhejiang University
Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain&Data Security
xiezl@zju.edu.cn

Lidan Shou*
The State Key Laboratory of Blockchain and Data Security, Zhejiang University
Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain&Data Security
should@zju.edu.cn

Ke Chen
The State Key Laboratory of Blockchain and Data Security, Zhejiang University
Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain&Data Security
chenk@zju.edu.cn

Gang Chen
The State Key Laboratory of Blockchain and Data Security, Zhejiang University
Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain&Data Security
cg@zju.edu.cn

Abstract

Customizing Pretrained Models (PMs) for downstream tasks demands task-specific data, which burgeoning Web3 applications provide abundantly. Blockchain-powered federated learning (BCFL) offers a secure and privacy-preserving paradigm to customize PMs among *self-interested* and *untrusted* Web3 participants. Yet, existing BCFL algorithms face severe *efficiency* and *scalability* challenges when applied to large PMs, due to their substantial computational and memory requirements and the *hardware constraints* of many participants. This paper introduces DICE, a novel BCFL algorithm that addresses the efficiency and scalability challenges of customizing large PMs in Web3 via two sets of synergistic techniques: (i) a *dynamic progressive learning* scheme and a *model slice verification* scheme to enable a wider range of honest participants, including those unable to deploy the full model, to engage in customizing PMs, while preserving model utility; (ii) a *tree-structured committee consensus* protocol and an *efficiency-driven incentive* scheme to address the dominant bottleneck that hinders existing BCFL algorithms from scaling to a larger number of data holders for customizing data-hungry large PMs, while maintaining security. Theoretical analysis and extensive experimental results on customizing vision and language PMs demonstrate DICE's efficiency, effectiveness, and security.

CCS Concepts

• **Computing methodologies** → **Machine learning**.

Keywords

Federated Learning, Pretrained Models, Blockchain, Web3

ACM Reference Format:
Hong Lin, Zhongle Xie, Lidan Shou, Ke Chen, and Gang Chen. 2026. DICE: Decentralized Incentive-driven Federated Learning for Efficient & Scalable Customization of Pretrained Models in Web3. In *Proceedings of the 35th ACM International Conference on Information and Knowledge Management (CIKM '26), November 07–11, 2026, Rome, Italy*. ACM, New York, NY, USA, 27 pages. https://doi.org/10.1145/3799682.3840574

*Lidan Shou is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
CIKM '26, Rome, Italy
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2539-5/2026/11
https://doi.org/10.1145/3799682.3840574

1 Introduction

Pretrained AI Models (PMs) and Web3 are converging to create more intelligent, automated, and efficient decentralized systems. These PMs require task-specific **customization** through further pretraining or fine-tuning before serving downstream tasks [12, 89, 114]. Meanwhile, burgeoning Web3 applications generate vast amounts of decentralized data [8, 29, 63, 92, 131], which offer immense potential to customize PMs. For instance, platforms such as MedSynapse [68] and Mind Network [74] produce terabytes of patient health data spread across millions of peer-to-peer devices. These data are valuable for customizing pretrained BERT [24] for disease diagnosis and treatment. However, aggregating private data on a single platform, whether centralized or decentralized, may violate increasingly stringent data protection regulations [101]. Moreover, Web3 participants are *self-interested* and *untrusted*: they are reluctant to contribute resources without monetary incentives and may disrupt customization tasks if malicious behavior is profitable [52, 58, 119].

Blockchain-powered Federated Learning (BCFL) [51, 55, 64, 75, 77, 82, 130] has emerged as a promising paradigm to support secure and privacy-preserving customization of a global model shared among Web3 participants. As shown in Fig. 1, a typical BCFL-based customization pipeline involves three participant roles: (i) a *model requester*, e.g., a medical institution, who needs a customized AI model and pays rewards to engage the other participants in customization; (ii) *workers*, e.g., MedSynapse's health data holders, who

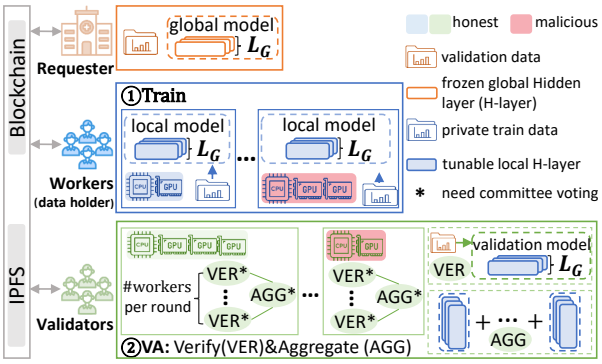


Figure 1: A general BCFL paradigm for customizing BERT-Large (with $L_G = 24$ H-layers, 340 million params), required by a medical institution for disease diagnosis, using health data in MedSynapse.

Table 1: Per-round memory costs of a worker (MC_w) and a validator (MC_v) in Fig. 1 under typical algorithms [57, 128].

Batch Size	8	16	32	64	**# Workers**	10	40	70	100
MC_w **(GB)**	11	17	30	59	MC_v **(GB)**	18	71	125	170

- These statistics are measured on NVIDIA A100 (PyTorch 2.6, CUDA 12.8) using the 2018n2c2 NER dataset [49], FP32 precision, Adam optimizer, sequence length 512, and batch size 8 unless otherwise stated.
- MC_v is reported as the total GPU memory usage across devices participating in the computation and corresponds to concurrent verification of all worker updates, without memory-swapping techniques that can increase verification latency [125].

train models using private data and submit *updates* (model parameters updated in training); (iii) *validators*, who verify and aggregate those updates. BCFL algorithms generally coordinate participants via a blockchain network and proceed in *iterative rounds of training and verification & aggregation (VA) steps*. This paradigm relies on two critical components: (i) *a malicious participant handling scheme* [57, 128] that requires validators to verify worker updates and execute a consensus protocol to ensure security, and (ii) *an incentive scheme* [123] that encourages self-interested entities to participate and behave honestly, typically via monetary rewards, e.g., cryptocurrency [43].

However, existing BCFL algorithms are primarily designed for small models (≤ tens of millions of parameters) [13, 57, 66], facing critical challenges in efficiency and scalability when applied to large PMs (typically ≥ 100 million parameters):

C1. Per-participant computational and memory overhead. Existing BCFL algorithms require every worker to perform training on the full model, and require every validator to perform multi-update verification [13, 55, 57, 66, 124]. As shown in Table 1, even for BERT-Large, an early-stage large pretrained model, these requirements already exceed the memory capacity of many commodity GPUs. Consequently, a large fraction of honest devices are excluded from effective participation.

C2. System-level scalability bottleneck. Since each validator in existing BCFL systems is required to verify all worker updates in every round [51, 77, 82], the per-validator overhead in those systems grows linearly with the worker scale (see Table 1). This requirement creates a critical bottleneck that can increase consensus latency and restrict participation as the worker population grows, preventing those systems from scaling to more workers to improve the performance of customizing large PMs, especially those data-hungry PMs.

C3. Limitations of existing efficiency techniques. Prior methods for efficient PM customization, such as LoRA [41], BitFit [10], and other parameter-efficient tuning (PEFT) techniques [129], can reduce worker-side costs but fail to address validator-side overhead. While model compression techniques such as distillation [38], pruning [56], and quantization [36] can reduce validator-side costs, they might sacrifice model quality [106, 125]. Critically, none of these techniques overcomes the above scalability bottleneck.

To address these challenges, this paper proposes `DICE`, a novel Decentralized InCEntive-driven BCFL algorithm for efficient, scalable, and secure customization of PMs in Web3. `DICE` supports lightweight training and verification of a large PM by two techniques built upon insights from existing efficient customization solutions: (i) a *Dynamic Progressive Learning (DPL)* scheme to reduce worker-side hardware resource requirements while maximizing the adaptability of PMs; and (ii) a *Model Slice Verification (MSV)* scheme to enable honest yet hardware-constrained participants to engage as validators, increasing the pool of potential validators and thus making malicious behavior less likely to succeed. More importantly, `DICE` breaks the linear dependency between per-validator workload and worker scale by two core innovations: (i) a *Tree-structured Committee Consensus (TCC)* protocol that decomposes the monolithic VA task into tree-structured sub-tasks and employs selective committee voting, thereby reducing the per-validator mandatory workload while maintaining security; and (ii) an *Efficiency-Driven Incentive (EDI)* scheme that complements TCC by penalizing conflict generators, ensuring rational validators do not undermine TCC's efficiency gains. Jointly, TCC and EDI resolve the scalability bottleneck that prior BCFL systems cannot address even when integrated with efficient model customization techniques, while preserving robust security guarantees.

The contributions of this paper are outlined as follows:

• **Systematic Problem Analysis and Novel Algorithm.** We systematically analyze the efficiency and scalability challenges of customizing large PMs within existing BCFL systems, and propose `DICE`, a novel algorithm that significantly lowers the entry barriers of customization tasks, enabling a wider range of honest participants to contribute effectively. (§3)

• **Novel Technical Designs.** We introduce two sets of novel techniques in `DICE`: DPL&MSV to support lightweight training and verification of large PMs while preserving model utility; TCC&EDI to address the dominant bottleneck that hinders prior BCFL systems from scaling to a larger number of workers without compromising security. (§3.2–§3.3)

• **Theoretical Guarantees.** We provide a rigorous theoretical analysis demonstrating that `DICE` lowers the participation barrier for both workers and validators and allows the per-validator workload required for consensus to scale *sub-linearly* with the worker scale, while bounding the success probabilities of malicious behaviors. (§4)

• **Experimental Validation.** We conduct extensive experiments on customizing vision and language PMs to show that DICE significantly reduces both worker-side and validator-side costs compared to baselines, delivers competitive final model performance, and maintains robust resistance against malicious actors. Additional experiments demonstrate that DICE is compatible with existing efficient customization techniques, accommodating diverse device constraints and model performance requirements without compromising security. (§5)

2 Preliminaries

We begin by presenting the system model, threat model, and foundational techniques from prior BCFL work on which DICE builds.

2.1 System Model

2.1.1 Participants. Similar to existing BCFL designs [57, 124], DICE involves three types of participants: model *requesters*, *workers*, and *validators*, as introduced in §1. Requesters need abundant data held privately by workers to customize PMs [14] for superior performance on downstream tasks, and possess powerful hardware to perform computation on large PMs. However, workers and validators may lack the hardware capacity to deploy a full model.

2.1.2 Coordination and Communication. DICE adopts a peer-to-peer *blockchain network* to coordinate participants. To initialize a customization task, the requester creates a smart contract that records task specifications (e.g., global model, validation dataset, algorithm hyperparameters) and manages cryptocurrency to reward honest participants who strictly follow the algorithm's logic [97, 103, 124, 128]. Next, workers and validators join the task by registering their unique identifiers, usually their network addresses, on this contract. During task runtime, all participants communicate with the smart contract to exchange data. For cost-efficient data exchange, we follow previous studies [18, 57, 124, 126] to integrate a *secure messaging scheme*. This scheme stores large-scale data that are too expensive to store on the blockchain (e.g., model parameters) in an additional storage system such as IPFS, persists data hashes and file handles in smart contracts, and ensures data availability and integrity by excluding participants from reward distribution if their file handles are unreachable or if their hashes, recomputed upon file retrieval, do not match the on-chain records. All messages are transmitted over authenticated channels using digital signatures, and messages sent repeatedly by honest participants are eventually delivered, though delays may be unbounded.

2.2 Threat Model

Threats to BCFL primarily come from two aspects: the coordination and communication infrastructures and algorithm participants [34].

2.2.1 Trust Assumptions. Following existing work [4, 5, 57, 65, 117, 124], we treat the blockchain network and IPFS as reliable underlying infrastructures and consider algorithm participants as *untrusted*, *self-interested*, yet *rational* players. Rational requesters have no incentive to deviate from the algorithm, as that would undermine their goal of customizing high-quality models and engaging other participants. We thus focus on threats from workers and validators.

2.2.2 Adversary Capacities. A malicious worker or validator may deviate from the algorithm in three forms: (i) *Poisoning*: computing and submitting low-quality updates or corrupted VA results to degrade model performance [19, 23, 45, 57, 69, 86]; (ii) *Fraud* (free-riding): copying and submitting honest participants' updates or VA results to claim undeserved rewards at a total computational cost substantially lower than that of executing the prescribed computation, potentially with negligible impact on model performance [20–22, 52, 57, 119, 122]; (iii) *No response*: failing to submit results due to network issues, crashes, etc., which may stall the algorithm [57]. Malicious participants can launch attacks independently or collude, and may further amplify their impact via blockchain-layer malicious behaviors such as Sybil attacks [26] (controlling multiple fake identities) or DDoS attacks [34] (disrupting message delivery). Privacy inference attacks and their mitigation (e.g., via differential privacy) [34] are orthogonal to the algorithm-deviation threats we consider and outside the scope of this study.

2.3 Foundational Techniques Against Threats

DICE adopts a malicious participant handling scheme architecturally similar to existing BCFL systems [55, 57, 124, 128]. This scheme relies on validators to execute a *worker verification* step to detect active malicious workers and a *cross-validator consensus* protocol to prevent active malicious validators. Consensus results produced by honest validators ensure that worker updates are verified and aggregated according to the prescribed rules. To handle non-responsive participants, this scheme integrates a *timeout* mechanism. Below, we review these base techniques and illustrate how DICE adopts or innovates beyond them.

2.3.1 Handling Malicious Workers. Worker verification aims to detect and reject invalid updates from malicious workers. Existing approaches fall into two categories: scoring-based (e.g., empirical thresholds on validation performance) and cryptography-based (e.g., zero-knowledge proofs) [34, 77]. DICE's MSV scheme builds upon the former category (since the latter cannot directly assess model quality) while remaining compatible with the latter. Unlike existing scoring-based schemes that require full model verification, MSV is specifically designed to enable validators who cannot deploy full models to engage. This increases the total resources that can be leveraged for malicious behavior prevention and raises the difficulty of successful attacks.

2.3.2 Handling Malicious Validators. Consensus protocols are the core component for defending against malicious validators. These protocols generally fall into three paradigms [130]: proof-based [71, 87], committee-based [3, 54], and hybrid. DICE's TCC protocol builds upon the hybrid paradigm that employs *economic staking* as proof of eligibility to participate and *committee voting* to determine the valid results [57, 124]. This paradigm offers superior resource efficiency over proof-based alternatives, and unlike committee-based paradigms deployed in permissioned environments, enables open participation in customization tasks for anyone willing to contribute data or hardware resources. In contrast to existing protocols where each validator must process all worker updates, TCC and associated EDI are designed to break the linear dependency

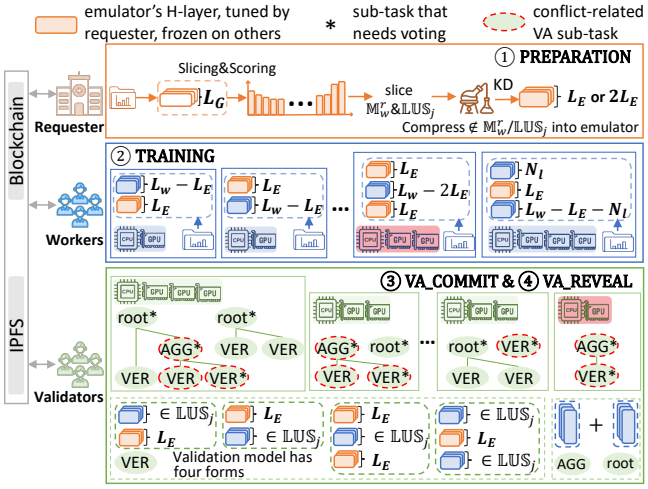


Figure 2: The overview of DICE.

between per-validator workload and worker scale, allowing DICE to scale to more workers for better customization performance.

Standard consensus protocols may lack inherent mechanisms that verify the authorship of the received results to prevent validator fraud attacks. To close this gap, a common solution [57, 62, 100] is to integrate a *cryptographic commitment scheme* into the consensus process. This scheme requires validators to submit local computation results in two phases: *committing* and *revealing*. When committing, validators submit *fingerprints* and *commitments* generated by hashing raw results. Only after a consensus fingerprint is determined will validators reveal the complete raw results. DICE adopts this scheme with a slight modification to accelerate TCC.

2.3.3 Handling Non-Responsive Participants. DICE inherits a timeout scheme from prior work [57] to prevent deadlock caused by non-responsive participants. This scheme sets a timeout for each stalling-prone task state, dynamically adjusting it based on the moving average of each state's actual execution time, and assumes that at least one honest participant who has access to the task's smart contract remains active in each BCFL task. When any state exceeds its timeout, any active participant can invoke the smart contract to terminate the task and let the model requester decide whether to restart the task or just acquire the global model customized so far and reclaim the remaining funds. The scheme is fully compatible with DICE's novel designs and thus is directly incorporated.

3 The DICE Algorithm

This section presents our DICE algorithm. §3.1 first provides a high-level overview of DICE's workflow. Then, §3.2–§3.3 introduce two sets of techniques that DICE utilizes to improve efficiency and scalability while maintaining effectiveness and security of model customization in Web3. Table 2 summarizes the main notations used in this paper.

3.1 Overview

Fig. 2 presents an overview of DICE for customizing the PM, which comprises an embedding layer, a sequence of intermediate hidden layers (H-layers), and an output layer. DICE executes every customization task in iterative rounds. Each round contains a *training* step, where workers produce updates for a global model, and a *VA* step, where validators verify worker updates and aggregate qualified updates to produce a new global model. In the training step, DICE uses a DPL scheme to enable hardware-constrained workers, especially those having abundant task-specific data, to engage. This scheme divides the training step into two states: PREPARATION and TRAINING. In the VA step, DICE adopts MSV, TCC, and EDI to reduce validator-side costs while maintaining security. TCC, integrated with a cryptographic commitment scheme, structures the VA step into VA_COMMIT and VA_REVEAL states.

Algorithms 1–2 present core procedures of DICE in four states. In PREPARATION, the start state of each round, the requester prepares necessary materials for the subsequent lightweight training and verification based on our DPL and MSV schemes (Alg. 1 lines 1–8, Alg. 2 lines 1–4). RequesterOnchainProcedure() changes the task state to TRAINING upon receiving those materials. Then, workers produce updates for the global model cost-effectively (Alg. 1 lines 9–13, Alg. 2 lines 5–9). After a certain number of workers have submitted updates, WorkerOnchainProcedure() creates a VA task forest based on our TCC scheme and changes the task state to VA_COMMIT. During VA_COMMIT, validators produce and submit fingerprints and commitments of VA results (Alg. 1 lines 14–22). ValidatorOnchainProcedure() checks whether validators reach

Table 2: Notation Summary.

Symbol	Definition
r, R, η	global round index; total number of rounds; learning rate
w, v	worker's and validator's addresses on the blockchain network ♠
$\mathbb{W}_{all}, \mathbb{V}_{all}$	the complete sets of eligible workers and eligible validators
$\mathbb{AW}^r$	workers that have submitted updates in round r
$\mathbb{W}^r$	scheduled workers; replaced by $\mathbb{AW}^r$ when training completes
$\mathbb{V}^r$	validators that submit VA results in round r
θ^r, θ^r_w	global model parameters and worker w's update in round r
$\mathbb{VAR}_v$	the set of results produced by validator v for a VA sub-task
L_G, L_E	H-layer counts in the global model and the emulator
L_w, L_v	deployable H-layer counts on worker w and validator v
$\mathbb{M}^r_w$	model slice selected to be trained on worker w in round r
$\mathbb{LUS}_j$	the j-th sub-slice of $\bigcup_{w\in\mathbb{W}^r}\mathbb{M}^r_w$
N_{lus}	number of slices $\mathbb{LUS}_j$
$\theta^{r-1}_{\mathbb{LUS}_j}$	global model parameters corresponding to $\mathbb{LUS}_j$ at the beginning of round r
$\theta^{r-1}_{\mathbb{LUS}_j}[l]$	global model parameters corresponding to layer l in $\mathbb{LUS}_j$ at the beginning of round r
$\theta^r_{*,\mathbb{LUS}_j}$	parameters corresponding to $\mathbb{LUS}_j$ from any specific worker or set of workers
$\theta^r_{*,\mathbb{LUS}_j}[l]$	parameters corresponding to layer l in $\mathbb{LUS}_j$ from any specific worker or set of workers
N_c, N_t	integer thresholds used in the TCC protocol
$N_{v,t}$	total number of distinct VA sub-tasks completed by v
$N^{correct}_{v,t}$	number of distinct VA sub-tasks correctly completed by v
$N^{pre}_{v,t}$	number of distinct VA sub-tasks completed by v and confirmed on chain when the first controversial sub-task appears
$N^{post}_{v,t}$	number of distinct VA sub-tasks correctly completed by v after conflict appears
E_{total}	total amount of security deposit staked by all validators
E_c	expected amount of stake selected for committee formation
ϵ_j, ϵ_c	model performance threshold; voting threshold
WR, VR	rewards for workers and validators in each round
$\|\cdot\|, \cdot \| \cdot$	element count in a set; hash input concatenation
$\Omega(\cdot), O(\cdot)$	asymptotic lower bound; asymptotic upper bound
$\Theta(\cdot)$	asymptotically tight bound
$\beta_{\text{hist}}, \delta$	history-window length; numerical stabilizer

♠ Each worker/validator is represented by its unique address.

Algorithm 1 DICE's off-chain procedures, i.e., computations executed locally by each participant.

1: **function** RequesterOffchainProcedure( ) ▷ §3.2
2: $step, r, r_g \leftarrow$ReadBlockchain()
3: **if** $step \neq$PREPARATION **or** $r \leq r_g$ **then return**
4: $\{\mathbb{M}^r_w | w \in \mathbb{W}^r\}, \{\mathbb{LUS}_j | j \in [0, N_{lus})\} \leftarrow$SliceSelection()
5: $\mathbb{M} \leftarrow \{\mathbb{M}^r_w | w \in \mathbb{W}^r\}; \mathbb{LUS} \leftarrow \{\mathbb{LUS}_j | j \in [0, N_{lus})\}$
6: $\{\theta^r_{\notin \mathbb{M}^r_w} | w \in \mathbb{W}^r\}, \{\theta^r_{\notin \mathbb{LUS}_j} | j \in [0, N_{lus})\} \leftarrow$ModelDerivation()
7: $\mathbb{EM}_w, \mathbb{EM}_v, \mathbb{GM} \leftarrow$RequesterIPFSStore()
8: RequesterOnchainProcedure($\mathbb{GM}, \mathbb{EM}_w, \mathbb{EM}_v, \mathbb{M}, \mathbb{LUS}$)
9: **function** WorkerOffchainProcedure( ) ▷ §3.2
10: $step, r, r_w \leftarrow$ReadBlockchain()
11: **if** $step \neq$TRAINING **or** $r \leq r_w$ **then return**
12: $\{\theta^r_{w,\mathbb{LUS}_j} | j \in [0, N_{lus})\} \leftarrow$ DPLtrain()
13: $\mathbb{LU} \leftarrow$WorkerIPFSStore(); WorkerOnchainProcedure($\mathbb{LU}$)
14: **function** ValidatorOffchainProcedure( ) ▷ §3.2–3.3
15: $step \leftarrow$ReadBlockchain()
16: **if** $step \notin${VA_COMMIT, VA_REVEAL} **then return**
17: $\mathbb{T}_{VA}, \mathbb{T}_v \leftarrow$ReadBlockchain(); ▷ $\mathbb{T}_v$ stores sub-tasks done by v
18: **if** $step =$VA_COMMIT **then**
19: **while** $step =$VA_COMMIT **and** $|\mathbb{T}_v| < |\mathbb{T}_{VA}|$ **do**
20: $\mathbb{RES}^{commit}_v \leftarrow$MSVorAGG()
21: ValidatorOnchainProcedure($\mathbb{RES}^{commit}_v$)
22: $step, \mathbb{T}_v \leftarrow$ReadBlockchain()
23: **else if** $|\mathbb{T}_v| > 0$ **then**
24: $\mathbb{RES}^{reveal}_v \leftarrow$RevealResPacking()
25: ValidatorOnchainProcedure($\mathbb{RES}^{reveal}_v$)

Algorithm 2 DICE's on-chain procedures, programmed in the smart contract and executed on the blockchain network.

1: **function** RequesterOnchainProcedure($\mathbb{GM}, \mathbb{EM}_w, \mathbb{EM}_v, \mathbb{M}, \mathbb{LUS}$)
2: **if** $step \neq$PREPARATION **or** $r_g \geq r$ **then return**
3: RecordRequesterRes($\mathbb{GM}, \mathbb{EM}_w, \mathbb{EM}_v, \mathbb{M}, \mathbb{LUS}$)
4: $r_g \leftarrow r$; $step \leftarrow$ TRAINING
5: **function** WorkerOnchainProcedure($\mathbb{LU}$)
6: **if** $step \neq$TRAINING **or** $r_w \geq r$ **then return**
7: $r_w \leftarrow r$; $\mathbb{AW}^r \leftarrow$RecordWorkerRes($\mathbb{LU}$);
8: **if** $\mathbb{AW}^r = \mathbb{W}^r$ **then** ▷ sufficient active workers
9: $\mathbb{W}^r \leftarrow \mathbb{AW}^r$; CreateVATaskTree(); $step \leftarrow$VA_COMMIT ▷ §3.3
10: **function** ValidatorOnchainProcedure($\mathbb{RES}_v$)
11: **if** $step \notin$ {VA_COMMIT, VA_REVEAL} **then return**
12: **if** $step =$VA_COMMIT **then** OnchainCommit($\mathbb{RES}_v$)
13: **else** OnchainReveal($\mathbb{RES}_v$)
14: **function** OnchainCommit($\mathbb{RES}_v$) ▷ §3.3
15: $tid, \mathbb{VAR}^{\text{commit}}_v, FP_v, CC_v, \pi_v, hash_v \leftarrow \mathbb{RES}_v$
16: **if** not Record($tid, \mathbb{VAR}^{\text{commit}}_v, FP_v, CC_v, \pi_v, hash_v$) **then return**
17: **if** NeedCommitteeVoting(tid) **then**
18: Voting($tid, \mathbb{VAR}^{\text{commit}}_v, FP_v, CC_v, \pi_v, hash_v$)
19: **else**
20: CheckConflict($tid, \mathbb{VAR}^{\text{commit}}_v, FP_v, CC_v, \pi_v, hash_v$)
21: **if** $|\mathbb{T}_{VA}| - |\mathbb{CR}| \neq 0$ **then** ▷ check global consensus
22: **if** $|\{v | v \in \mathbb{V}_{all}, |\mathbb{T}_v \backslash \mathbb{CR}| \geq \lceil \frac{|\mathbb{T}_{VA}| - |\mathbb{CR}|}{|\mathbb{T}_{VA}|} N_t \rceil\}| < N_c$ **then**
23: **return**
24: **if** ($|\mathbb{T}_{VA}| = |\mathbb{CR}|$ **or** SupplementaryCheck()) **and**
25: $|\mathbb{CONSENSUS}^r| \geq |\mathbb{T}_{VA}|$ **then** ▷ reach consensus
26: $step \leftarrow$VA_REVEAL
27: **function** OnchainReveal($\mathbb{RES}_v$) ▷ §3.3
28: $\{< tid, \mathbb{VAR}^{\text{reveal}}_v > | tid \in \mathbb{T}_v\} \leftarrow \mathbb{RES}_v$
29: QualificationCheck($\{< tid, \mathbb{VAR}^{\text{reveal}}_v > | tid \in \mathbb{T}_v\}$)
30: **if** all($\{|\mathbb{T}_v| = 0 | v \in \mathbb{V}_{all}\}$) **then**
31: EDI(); OnchainPreparation(); $step \leftarrow$PREPARATION

consensus on the fingerprints of all VA sub-tasks (Alg. 2 lines 16–25), and if so, transitions the task state to VA_REVEAL. Then, validators reveal the complete VA results, which are checked by the on-chain procedure (Alg. 1 lines 24–25, Alg. 2 lines 28–29). Finally, EDI() rewards honest participants and penalizes malicious validators (Alg. 2 line 31); then the customization task proceeds to the next round (line 31).

3.2 Lightweight Training and Verification powered by DPL and MSV

This section introduces lightweight training and verification procedures powered by our DPL and MSV schemes. The former aims to enable constrained workers to produce updates of a large PM and customize all necessary layers of the PM on worker data to maximize the PM's adaptation ability. The latter aims to enable a wider range of validators, including those more hardware-constrained than workers, to verify updates, increasing the total hardware resources that can be leveraged to augment security. These objectives are achieved through the *Slice Selection* and *Large-to-Compact Model Derivation* procedures executed by the model requester. §3.2.1–§3.2.3 first illustrate how the requester executes these two procedures. Then, §3.2.4 describes the lightweight training and verification process on workers and validators. Finally, §3.2.5 discusses the design rationale behind our proposed schemes.

3.2.1 Slice Selection. In this procedure, the model requester selects a small slice of the global model for training on each worker based on the slice's importance score, and executes further-slicing for validators when necessary (Alg. 1 line 4). For each worker w capable of deploying the full global model, the requester selects all H-layers and the output layer to form the slice to be trained on this worker, denoted by $\mathbb{M}^r_w$.

For each worker w that cannot deploy the full global model, the requester divides the global model into a set of candidate slices $\mathbb{M}_{w,i}$ defined in Eq. 1 using this worker's hardware capacity L_w reported at registration. Each slice contains a small number of H-layers. Empty $\mathbb{M}_{w,i}$ are excluded from the subsequent slice selection process.

$$\mathbb{M}_{w,i} = \begin{cases} \{1, \cdots, L_w - L_E\} & \text{if } i = 0 \text{ (shallow)} \\ \{1 + L_E + i, \cdots, L_w - L_E + i\} & \text{if } i \in [1, N'_{\text{slice}}) \text{ (middle)} \\ \{L_G - (L_w - L_E - 1), \cdots, L_G\} & \text{if } i = N'_{\text{slice}} \text{ (top)} \\ \{1, \cdots, L_w - L_E - (i - N'_{\text{slice}}), & \\ \ L_G - (i - N'_{\text{slice}} - 1), \cdots, L_G\} & \text{if } i \in (N'_{\text{slice}}, N_{\text{slice}}] \text{ (combined)} \end{cases} \quad (1)$$

Here, the numbers in $\mathbb{M}_{w,i}$ denote the indices of global H-layers. L_G is the total number of H-layers in the global model. L_E is the number of H-layers in an emulator. L_w is the number of H-layers deployable on worker w, $N'_{\text{slice}} = L_G - L_w$, $N_{\text{slice}} = L_G - L_E - 1$.

Then, the requester computes each slice's importance score denoted by $Score_{w,i}$. Our DPL scheme defines $Score_{w,i}$ by Eq. 2, where $s^r_{\mathbb{M}_{w,i}}$ is the performance of the global model after a slice $\mathbb{M}_{w,i}$ is tuned, $div^r_{\mathbb{M}_{w,i}} = 1 - cos_sim(\theta^{r-1}_{\mathbb{M}_{w,i}}, \hat{\theta}^r_{\mathbb{M}_{w,i}})$ is the divergence of the slice parameters before and after tuning, $max_s =$

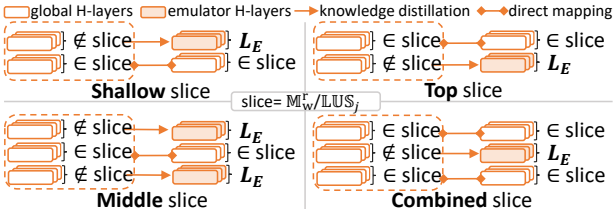


Figure 3: Emulator generation given slice $\mathbb{M}_w^r$/$\mathbb{LUS}_j$.

$\max(\{s^r_{\mathbb{M}_{w,i}} | i \in [0, N_{\text{slice}}],\ \mathbb{M}_{w,i} \neq \emptyset\})$, $max_d = \max(\{div^r_{\mathbb{M}_{w,i}} | i \in [0, N_{\text{slice}}],\ \mathbb{M}_{w,i} \neq \emptyset\})$, and $\delta = 10^{-12}$ prevents division by zero. Each candidate slice is evaluated independently from the same round-start global model, using the validation data. Only its selected H-layers and the output layer are tuned; all other parameters remain frozen.

$$Score_{w,i} \leftarrow \frac{s^r_{\mathbb{M}_{w,i}}}{max_s + \delta} + \frac{div^r_{\mathbb{M}_{w,i}}}{max_d + \delta} \tag{2}$$

This design prioritizes training slices that yield greater performance improvement and exhibit more substantial parameter updates. The model requester selects $\mathbb{M}_{w,i}$ with the highest score and combines it with the output layer to form the final slice $\mathbb{M}_w^r$ to be trained on worker w.

Finally, the model requester executes further-slicing based on our MSV scheme to enable more validators to participate in verifying worker updates. The requester checks whether the union $\mathbb{U}^r = \bigcup_{w \in \mathbb{W}^r} \mathbb{M}_w^r$ can form a validation model with at most L_{vm} H-layers under the model construction scheme introduced in §3.2.2, where L_{vm} is the number of H-layers that can be deployed on the most hardware-constrained validator. If so, it sets $N_{lus} = 1$ and $\mathbb{LUS}_0 = \mathbb{U}^r$. Otherwise, the requester partitions $\mathbb{U}^r$ into disjoint sub-slices $\{\mathbb{LUS}_j\}_{j=0}^{N_{lus}-1}$, where N_{lus} is the number of sub-slices produced based on the following partition conditions: (i) The output layer forms a separate sub-slice. (ii) The H-layer sub-slices cover all H-layers in $\mathbb{U}^r$, and each retains a shallow, top, middle, or combined structure as shown in Fig. 3. (iii) Each sub-slice, together with its emulator, must form a validation model containing at most L_{vm} H-layers. The above design requires each participant to support the deployment of at least $3L_E$ H-layers. Participants below this capacity are not admitted as workers or validators in DICE.

For every slice $\mathbb{M}_w^r$ and $\mathbb{LUS}_j$, the model requester compresses the global H-layers excluded from that slice into an emulator of a compact model by ModelDerivation() (Alg. 1 line 6).

3.2.2 Large-to-Compact Model Derivation. In this procedure, the model requester derives compact models from the global model for both workers and validators (Alg. 1 line 6). Each compact model retains the same embedding and output layers as the global model but contains a reduced number of H-layers. These compact models are designed to allow workers to efficiently generate updates for $\mathbb{M}_w^r$, and to enable validators to verify each update at a manageable cost. Motivated by offsite tuning [110], we insert a lightweight *emulator* into each compact model to ensure that each worker's update and the corresponding verification results closely approximate those obtained by training or verifying with the full model, thereby achieving performance comparable to customizing the full model.

As presented in Fig. 3, an emulator is obtained by compressing the global H-layers not in $\mathbb{M}_w^r$ (or $\mathbb{LUS}_j$) through knowledge distillation (KD) [38, 110] using the validation data. It contains L_E H-layers by default, or $2L_E$ H-layers for a middle slice, with $L_E = 1$ in our default setting. For $\mathbb{LUS}_j$ with only the output layer, all global H-layers are distilled into an emulator containing L_E H-layers. For $\mathbb{M}_w^r$ or $\mathbb{LUS}_j$ containing all global H-layers, no emulator is needed.

3.2.3 Runtime Mechanics. We now describe how the model requester executes the above procedures with two runtime mechanisms incorporated in DPL and MSV to balance model performance and computation efficiency.

(i) *Periodic Slice Reselection and Emulator Regeneration.* Rather than executing slice selection and large-to-compact model derivation in every round, we design a more cost-effective periodic mechanism that avoids unnecessary slice reselection or emulator regeneration and decouples the update frequency of the selected slices and the emulators based on their respective roles.

Under this periodic mechanism, the requester re-computes slice importance scores and selects new slices to train on workers only when the average performance of the global model over the latest $\epsilon_r \in \mathbb{Z}^+$ rounds does not show distinct improvement. Otherwise, the requester skips slice selection and proceeds directly to the model derivation step, reusing the same slices as in the previous round. The rationale is that slice selection determines which part of the model to train, and a performance plateau signals the exhausted utility of further training the current selected slice and the necessity of exploring a different slice.

The requester generates an emulator when a slice/sub-slice is first assigned or its structure changes. For an unchanged slice/sub-slice, the emulator is regenerated only when the output discrepancy between the full global model and the compact model exceeds $\epsilon_{em} \in \mathbb{R}^+$. The rationale is that the emulator's role is to approximate the input-output mappings of the global H-layers excluded from the compact model, and a large output discrepancy implies the ineffectiveness of the current emulator for such approximation and the necessity of emulator regeneration.

With the above designs, the model requester re-selects slices and regenerates emulators in response to performance stagnation or model drift, respectively, thereby balancing model adaptability with computational efficiency.

(ii) *Adaptive Mode Switching.* In addition to the above periodic mechanism, DPL further incorporates an adaptive mode switching mechanism to accommodate heterogeneous hardware capabilities among workers. Under this mechanism, the requester dynamically switches between two operational modes of DPL. One is *Personalized* (P-DPL), where slices are selected per worker to maximize the utilization of available hardware, as described in §3.2.1. The other is *Unified* (U-DPL), where the same slice is selected for all workers by using a unified parameter $L_{wm} = \min(\{L_w | w \in \mathbb{W}_{all}\})$ to replace every worker's specific parameter L_w in §3.2.1. U-DPL is the default mode, since it requires the slice selection to be executed only once for all workers in any round and thus is cost-effective for requesters. The requester activates the P-DPL mode only if there exist hardware-rich workers (with $L_w > L_{wm}$) whose updates consistently achieve a performance score higher than $\frac{L_w}{L_{wm}} \cdot s_{wm}$, where

s_{wm} is the best performance score that hardware-constrained workers (i.e., with $L_w = L_{wm}$) have ever achieved in all past rounds. Once this condition ceases to hold, the requester switches DPL back to its default mode.

This adaptive mechanism is motivated by the observation that the benefit of personalization depends on the alignment between hardware capacity and local training quality. Specifically, assigning larger slices to train on hardware-rich workers that fail to produce well-performing updates could lead to overfitting on those workers, reducing the generality of the global H-layers that hardware-constrained workers do not update, thereby compromising the final global model performance. Thus, P-DPL is preferred only when capable workers demonstrate consistently superior local performance exceeding a dynamic threshold.

Together, the two runtime mechanisms enable DICE to adapt the requester-side operations to both the model's learning dynamics and the participants' resource heterogeneity, balancing the requester's overhead, the engagement of more honest but hardware-constrained participants, and model performance.

3.2.4 Lightweight Training and Verification. After the requester-side procedures complete, the worker constructs a compact local model for training, and the validator constructs a compact validation model for verification, as shown in Fig. 2. Each worker w executes DPLtrain() to produce the update for $\mathbb{M}_w^r$ and divide it into $\{\theta^r_{w,\mathbb{LUS}_j} | j \in [0, N_{lus})\}$ (Alg. 1 line 12). Each validator v executes MSVorAGG() to verify and aggregate $\theta^r_{w,\mathbb{LUS}_j}$ (Alg. 1 line 20). An update $\theta^r_{w,\mathbb{LUS}_j}$ will be considered valid only if it satisfies two conditions: (i) it contains at least one layer whose norm diverges from the historical norms of updates for that layer to pass the fraud check; (ii) its corresponding validation model performs better than a dynamically adjusted threshold ϵ_j to pass the poisoning check. When constructing $\theta^r_{w,\mathbb{LUS}_j}$'s corresponding validation model, the parameters submitted by w are used for layers trained by w. For H-layers in $\mathbb{LUS}_j$ that w did not train, and for the output layer when it is excluded from $\mathbb{LUS}_j$, the model uses the round-start global parameters. The remaining H-layers are replaced by the corresponding emulator.

The threshold ϵ_j is initialized by the model requester using the global model's validation score at the start of the first training round. For $r \geq 2$, let s_G^r denote the global model's validation score before training in round r, $\mathbb{S}_{\mathbb{LUS}_j}$ contain the consensus-confirmed validation scores of qualified updates for the current sub-slice $\mathbb{LUS}_j$ over the latest β_{hist} rounds, $s_{VM,\mathbb{LUS}_j}$ be the validation score of $\mathbb{LUS}_j$'s corresponding compact model at its most recent derivation, and $\overline{\Delta}^r = \frac{\sum_{q=\max(2,r-\beta_{\text{hist}}+1)}^{r}(s_G^q - s_G^{q-1})}{r-\max(2,r-\beta_{\text{hist}}+1)+1}$. The threshold is updated before training by Eq. 3. This rule adapts the acceptance threshold to the progress of global model performance and the actual quality of qualified updates.

$$\epsilon_j = \begin{cases} \min\left(s_G^r + \overline{\Delta}^r, 2\,\text{median}(\mathbb{S}_{\mathbb{LUS}_j}) - \min(\mathbb{S}_{\mathbb{LUS}_j})\right), & \mathbb{S}_{\mathbb{LUS}_j} \neq \emptyset \\ \min\left(s_G^r + \overline{\Delta}^r, s_{VM,\mathbb{LUS}_j}\right), & \text{otherwise} \end{cases} \tag{3}$$

3.2.5 Discussion. We finally discuss the design rationales of DPL and MSV. Both schemes are motivated by the resource asymmetry between the model requester and the other participants, particularly the workers and validators who cannot deploy full models. They leverage the relatively abundant hardware resources of the requester to select slices and generate emulators for compact model execution on constrained participants, enabling those participants to effectively contribute to customizing large PMs. By integrating two runtime mechanisms, namely (i) periodic slice reselection and emulator regeneration and (ii) adaptive mode switching, they slow down the growth of requester-side costs as the number of training rounds or the participant scale increases.

Meanwhile, the two schemes are designed to mitigate the potential biases introduced by using compact models in the training and verification stages, respectively. DPL mitigates training bias by dynamically adjusting the slice to be trained under heterogeneous hardware constraints among workers based on a slice importance score, which quantifies the expected utility of training a specific set of layers by the potential performance improvement and the magnitude of parameter updates. By selecting the slice with the highest score, DPL uses information in the specific task runtime to determine which part of the PM should be customized in each round, rather than following a fixed heuristic solution. MSV mitigates verification bias by employing a threshold ϵ_j, dynamically adjusted by the model requester based on the full global model's performance trend and actual distribution of the quality of worker updates, to compensate for systematic under- or over-estimation of worker update quality.

Furthermore, DPL and MSV jointly make poisoning attacks more difficult. Compared to full model training and verification, the two schemes restrict parameters that each malicious worker can modify in each round to its assigned slice and the output layer, and each worker's update still needs to pass corresponding quality checks. The eventual effect of a poisoned update depends on not only the honest workers' concurrent updates but also the requester's future decisions on slice selection, emulator updates, and validation threshold (ϵ_j) adjustments, all of which might not be reliably predicted at the time of attack injection. The tunability of ϵ_j also enables MSV to accommodate diverse and evolving adversarial behaviors, as the adjustment logic for ϵ_j can be refined when new attack strategies emerge.

3.3 Balancing Efficiency, Scalability, and Security by TCC and EDI

This section enhances the lightweight verification introduced in the last section with our TCC protocol and EDI scheme, two integrated techniques to further reduce per-validator workload requirement and improve the system's ability to scale to more workers while ensuring security. §3.3.1 first presents how TCC breaks the linear dependency between each validator's workload and worker scale while generating correct consensus. Then, §3.3.2 presents how EDI complements TCC to strengthen the balance between efficiency, scalability, and security. Finally, §3.3.3 discusses the synergistic relationship between TCC and EDI.

3.3.1 Consensus Generation. TCC generates consensus over the entire VA task by four critical steps: Task Decomposition, Result Committing, Selective Committee Voting, and Result Revealing.

Task Decomposition (before VA_COMMIT state). In this step, a monolithic VA task is decomposed into N_{VA} sub-tasks in a VA task forest, where $N_{VA} \leq (2|\mathbb{W}^r| - 1)N_{lus}$ and $\mathbb{W}^r$ is the set of active workers in round r (Alg. 2 line 9). As shown in Fig. 2, the forest comprises several task trees. The leaf node in each $Tree_j$ represents a VER sub-task verifying a worker's update of $\mathbb{LUS}_j$. The non-leaf node represents an AGG sub-task aggregating qualified updates from two child nodes. Executing each non-leaf sub-task has a strong dependency on the results of its two child sub-tasks. For example, an AGG sub-task requires the performance scores computed in its child sub-tasks to determine each update's weight in aggregation. It may also need the file handles of the partially aggregated updates to retrieve those updates for further aggregation. After the VA task forest is constructed on the blockchain, result committing starts.

Result Committing (during VA_COMMIT state). Each validator iteratively executes the VA sub-task randomly selected from the forest based on its local hardware condition and submits the fingerprint and commitment of this sub-task's results to the blockchain based on a cryptographic commitment scheme to prevent fraud (Alg. 1 lines 18–22). The set of results from a validator v for a VA sub-task, denoted by $\mathbb{VAR}_v$, is produced as follows.

- For any VER sub-task, the validator first verifies a worker's update $\theta^r_{w,\mathbb{LUS}_j}$ as illustrated in §3.2.4, producing the sub-task's intermediate result, i.e., the output of the validation model built upon $\theta^r_{w,\mathbb{LUS}_j}$ on the validation data, denoted by $V(\theta^r_{w,\mathbb{LUS}_j}, \mathbb{D}_{val})$. If the update is qualified, the validator then computes its weight in the final aggregated update, denoted by $\alpha_{w,\mathbb{LUS}_j}$. We define $\alpha_{w,\mathbb{LUS}_j}$ by the performance of $\theta^r_{w,\mathbb{LUS}_j}$ on the validation data, while remaining compatible with alternative aggregation weight definitions, e.g., weights based on worker data volume [57, 67] and contribution to aggregated updates [30, 95]. Finally, the validator packs the task input version (identified by $H(\theta^r_{w,\mathbb{LUS}_j})$), the intermediate results $\alpha_{w,\mathbb{LUS}_j}$ and $V(\theta^r_{w,\mathbb{LUS}_j}, \mathbb{D}_{val})$, the iteration index, and the sub-task identifier into the result set, where $H(\cdot)$ is a cryptographic hash function.
- For any AGG sub-task, the validator first aggregates the qualified model updates received from its two child nodes via Eq. 4–6. The coefficient $\frac{\beta_{\mathbb{W}_c,\mathbb{LUS}_j}}{\tau^l_{c1,c2}}$ ensures that, for each model layer in $\mathbb{LUS}_j$ with qualified worker updates, the root node of each task tree $Tree_j$ outputs the weighted average of the qualified worker updates containing that layer. During the aggregation process, the validator produces the sub-task's intermediate results: $H(\{(\theta^r_{\mathbb{W}_{c1},\mathbb{LUS}_j}[l], \theta^r_{\mathbb{W}_{c2},\mathbb{LUS}_j}[l], \theta^r_{\mathbb{W},\mathbb{LUS}_j}[l], \tau^l_{c1,c2}) \mid l \in \mathbb{LUS}_j\} \parallel \beta_{\mathbb{W}_{c1},\mathbb{LUS}_j} \parallel \beta_{\mathbb{W}_{c2},\mathbb{LUS}_j})$ and $digest_{|\mathbb{LUS}_j|}$ defined by $digest_i = H(digest_{i-1} \parallel \theta^r_{\mathbb{W}_{c1},\mathbb{LUS}_j}[l_i] \parallel \theta^r_{\mathbb{W}_{c2},\mathbb{LUS}_j}[l_i] \parallel \theta^r_{\mathbb{W},\mathbb{LUS}_j}[l_i])$ for $i = 1, \ldots, |\mathbb{LUS}_j|$, where $digest_0$ is a fixed initialization value and $l_i = \mathbb{LUS}_j[i]$ denotes the i-th layer in a predefined ordering of $\mathbb{LUS}_j$. Finally, the validator packs the task input version (identified by the two child-node update hashes $H(\theta^r_{\mathbb{W}_{c1},\mathbb{LUS}_j})$ and $H(\theta^r_{\mathbb{W}_{c2},\mathbb{LUS}_j})$ together with the corresponding $\beta_{\mathbb{W}_{c1},\mathbb{LUS}_j}$, $\beta_{\mathbb{W}_{c2},\mathbb{LUS}_j}$, and $\{\tau^l_{c1,c2} \mid l \in \mathbb{LUS}_j\}$), the above intermediate results, the aggregated update hash $H(\theta^r_{\mathbb{W},\mathbb{LUS}_j})$, the iteration index, and the sub-task identifier into the result set.

$$\beta_{\mathbb{W}_c,\mathbb{LUS}_j} = \begin{cases} \alpha_{w,\mathbb{LUS}_j}, & w \in \mathbb{W}_c, \quad \text{if } |\mathbb{W}_c| = 1, \\ 1, & \text{otherwise.} \end{cases} \tag{4}$$

$$\tau^l_{c1,c2} = \begin{cases} \sum_{w \in \mathbb{W}_{j,l}} \alpha_{w,\mathbb{LUS}_j}, & \text{if } |\mathbb{W}_{c1} \cup \mathbb{W}_{c2}| = |\mathbb{W}_j|, \\ 1, & \text{otherwise.} \end{cases} \tag{5}$$

$$\theta^r_{\mathbb{W},\mathbb{LUS}_j}[l] = \begin{cases} \sum_{c \in \{c1,c2\}} \frac{\beta_{\mathbb{W}_c,\mathbb{LUS}_j}}{\tau^l_{c1,c2}} \theta^r_{\mathbb{W}_c,\mathbb{LUS}_j}[l], & \tau^l_{c1,c2} > 0, \\ \theta^{r-1}_{\mathbb{LUS}_j}[l], & \text{otherwise.} \end{cases} \tag{6}$$

Here, $c1$ and $c2$ denote the two child nodes, and $\mathbb{W}_c$ is the set of workers whose updates are involved in any child node $c \in \{c1, c2\}$. The model parameters corresponding to child node c are denoted by $\theta^r_{\mathbb{W}_c,\mathbb{LUS}_j}$; they are the corresponding worker's update if $\mathbb{W}_c$ is a singleton, and otherwise are the partially aggregated update. Moreover, $\mathbb{W} = \mathbb{W}_{c1} \cup \mathbb{W}_{c2}$, $\mathbb{W}_j = \{w \in \mathbb{W}^r \mid \mathbb{M}^r_w \cap \mathbb{LUS}_j \neq \emptyset\}$ denotes the set of workers participating in the training of the parameters corresponding to the j-th sub-slice, and $\mathbb{W}_{j,l}$ is the set of workers included in $\mathbb{W}_j$ and with $\theta^r_{w,\mathbb{LUS}_j}$ containing the parameters of layer l. For any update that fails verification, its corresponding weight is set to zero. If the updates from two child nodes cover different model layers, they are aligned using zero-valued padding, and the padded entries are excluded from the normalized aggregation of the corresponding layers. If a task tree $Tree_j$ contains only one leaf node, that leaf node also serves as the root: its worker update is adopted for the layers it covers if it passes verification, while the round-start global parameters are retained for all remaining layers.

The commitment scheme in prior work [57] defines the fingerprint and commitment of each result set $\mathbb{VAR}_v$ as $FP_v = H(\mathbb{VAR}_v)$ and $CC_v = H(\mathbb{VAR}_v \parallel v)$, respectively. TCC extends this scheme by treating the partial raw results $\mathbb{VAR}^{\text{commit}}_v$ ($\subset \mathbb{VAR}_v$), together with FP_v, as the sub-task fingerprint on which validators must reach consensus. For any VER sub-task, $\mathbb{VAR}^{\text{commit}}_v$ contains only the weight $\alpha_{w,\mathbb{LUS}_j}$ and the task input version, i.e., the worker-update hash. For any AGG sub-task, it contains only the aggregated update hash $H(\theta^r_{\mathbb{W},\mathbb{LUS}_j})$ and the task input version identified by the two child-node update hashes and their corresponding coefficients. TCC also requires validators executing an AGG sub-task to submit the file handle of the aggregated update together with its fingerprint and commitment. These designs enable validators to reuse available descendant results when executing non-leaf sub-tasks without re-execution, thereby reducing individual workload and global consensus latency across the VA task forest.

For any given sub-task, honest validators will generate the same valid fingerprint from the same task input version under uniform computation, encoding, and serialization rules, whereas malicious validators may submit poisoned results. `ValidatorOnchainProcedure()` feeds validator v's submission into `OnchainCommit()` for fingerprint validity checking and consensus fingerprint generation (Alg. 2 lines 12, 16–25). This function involves two important checks:

(i) For any given sub-task, it checks whether the task input version declared by v belongs to the available input versions recorded on-chain for that sub-task.

(ii) For a non-leaf sub-task, it further checks the results of each descendant sub-task used by v: these results must match either the results previously submitted by v for that descendant sub-task or its adopted consensus results. It also checks whether the update hashes declared in the task input version match the corresponding descendant sub-tasks' results and whether the update's coefficients are consistent with those results and Eq. 4–5. These checks enforce the sub-task input provenance and dependency consistency while allowing validators to reuse available descendant sub-tasks' results and avoid redundant computation.

The submission is rejected if any of the above checks fails. After the submission passes the above checks, if the blockchain has already recorded a commitment from v for the same sub-task and input version, the original commitment and its corresponding fingerprint are retained, and neither replacement nor an additional submission record is accepted. Otherwise, the submitted fingerprint and commitment are recorded. The validity of the retained or newly recorded fingerprint is determined by TCC's selective committee voting strategy.

Selective Committee Voting (during `VA_COMMIT` state). Fig. 4 depicts TCC's selective committee voting strategy (Alg. 2 lines 17–20). Committees are formed only for root sub-tasks at the start of the consensus process (Fig. 4①) to prevent malicious validators from prematurely satisfying the global consensus conditions by using quickly forged results without actual execution of any sub-task. For a non-root sub-task, a committee is formed only when it becomes controversial (Fig. 4③). Following prior work [57], our committee formation requires each validator to stake at least $E_{\min}$ units of cryptocurrency as a security deposit upon joining a customization task. Committee members for each sub-task are randomly selected from all validators, and each committee votes to determine the consensus fingerprint of its assigned sub-task. A validator with a larger stake has a higher probability of being selected and a larger expected number of votes.

Since the input of a root or controversial non-leaf sub-task may change with its descendant results, TCC determines whether to elect another committee for such a sub-task according to whether the identities of the existing committee members have been disclosed. If a sub-task with an existing committee needs to process a new input version and some members of the committee intended for reuse have disclosed their identities by submitting fingerprints and commitments, a new committee is formed using its corresponding election index. Otherwise, the existing committee is reused. When processing a root or controversial sub-task, each committee is associated with a specific input version and votes on the outputs generated from that version.

To prevent malicious validators from repeatedly triggering committee elections for the same sub-task, TCC records the number of committee elections for each sub-task in a single iteration and sets an upper bound $K_{\text{election}} \in \mathbb{Z}^+$, including the initial election. To distinguish different sub-tasks and different elections of the same sub-task, the election seed is generated from the current-round random seed, the sub-task identifier, and the election index of that sub-task. The on-chain procedure increments the election index from 1 to at most K_{election}. If another committee election is required after this limit has been reached, the current consensus process is terminated and the customization task is paused. The round-start

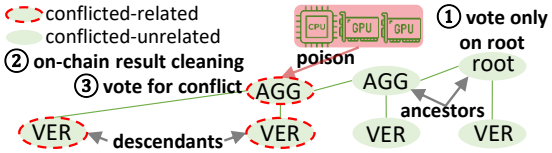


Figure 4: Selective committee voting.

global model parameters are retained, and the model requester determines whether to recruit validators again and restart the task.

For a root or controversial sub-task, `Voting()` is triggered (Alg. 2 line 18). This function first verifies validator v's membership in the committee corresponding to the sub-task and calculates its number of votes using the submitted proof π_v and hash value $hash_v$. For details of this verification process, please refer to prior work [32, 57]. Validator v's votes are counted only if one of the following eligibility requirements is satisfied: (i) For a non-controversial root sub-task, v's valid on-chain records must contain submissions for at least $\lceil \frac{N_{VA}-N_{CF}}{N_{VA}} N_t \rceil$ distinct conflict-unrelated sub-tasks, where N_{CF} is the number of conflict-related sub-tasks. If the total number of non-root conflict-unrelated sub-tasks is smaller than this threshold, v is instead required to complete and submit results for all non-root conflict-unrelated sub-tasks. (ii) For a controversial sub-task, v's valid on-chain records must contain its submissions for all descendants of that sub-task. Here, valid on-chain records form a subset of the complete on-chain submission history and contain submissions accepted under the preceding rules, including submissions consistent with the consensus of non-controversial sub-tasks and submissions used for committee voting. The first requirement, together with the supplementary check described below, supports detection of poisoned results in conflict-unrelated sub-tasks, whereas the second forces honest committee members to execute the descendant sub-tasks on which a controversial sub-task depends, thereby producing consistent and correct results for conflict-related sub-tasks.

If v's votes are eligible, `Voting()` records or updates its votes for the corresponding candidate fingerprint and updates the total votes received by each candidate. For each root or controversial sub-task, the highest-voted fingerprint receiving more than $E_c \epsilon_c$ votes in the committee, corresponding to this sub-task and its input version consistent with the consensus results of its dependent descendant sub-tasks, is adopted as this sub-task's consensus fingerprint. TCC allows validators to submit fingerprints and vote for any root or controversial sub-task before the consensus results of relevant descendant sub-tasks are available, but the consensus of any root or controversial sub-task is confirmed only after the above dependency requirement is satisfied.

For each non-root and non-controversial sub-task for which voting is unnecessary, `CheckConflict()` is invoked (Alg. 2 line 20). If no conflict is detected, the received fingerprint is directly adopted as the sub-task's consensus fingerprint. Otherwise, the sub-task is marked as controversial, and the sub-task and its descendants are marked as conflict-related, triggering a two-step resolution procedure. Firstly, the adopted consensus fingerprints of this controversial sub-task and its ancestors are revoked, and all submissions related to these sub-tasks are removed from the valid on-chain

records. The corresponding voting records, completed sub-task records, and counters related to the global consensus conditions are updated accordingly (Fig. 4②). The submitted fingerprints, commitments, submitter identities, and task input versions remain in the complete on-chain submission history for duplicate-commitment checking and subsequent reward or penalty determination. If a validator's submission for a corresponding sub-task is removed from the valid on-chain records, the validator must submit it again and pass the preceding checks before it can be restored to the valid records. This step prevents a controversial sub-task from compromising the correctness of its ancestors. If a descendant's results change after conflict resolution, its ancestors should be recomputed from the updated descendant results. After the above step, a new committee is randomly selected to vote on the consensus fingerprint of the controversial sub-task (Fig. 4③).

Global Consensus Determination. To determine whether the global consensus has been reached, the on-chain procedure first checks two conditions: (i) at least N_c validators each have valid on-chain records for at least $\lceil \frac{N_{VA}-N_{CF}}{N_{VA}} N_t \rceil$ distinct conflict-unrelated sub-tasks; and (ii) the consensus fingerprints of all VA sub-tasks have been confirmed. If no conflict-unrelated sub-task exists, the first condition is waived, and global consensus is reached once the second condition holds.

Otherwise, once the first condition is satisfied, a supplementary global consensus check is initiated to ensure that honest validators have submitted results for conflict-unrelated sub-tasks. The on-chain procedure selects N_c validators satisfying the first condition to form the checking member set $\mathbb{V}_c$. After this set has been fixed, a newly generated random value that was unpredictable before the set was determined is used to independently and uniformly sample $\lceil \frac{N_{VA}-N_{CF}}{N_{VA}} N_t \rceil$ conflict-unrelated sub-tasks without replacement for each validator $v \in \mathbb{V}_c$, forming v's supplementary checking set $\mathbb{CHECK}_v^r$. Here, N_{CF} is the number of conflict-related sub-tasks at the time of sampling. Each checking member completes the sampled sub-tasks and submits the corresponding fingerprints and commitments. If the valid on-chain records already contain a submission by that member for the same input version, the on-chain procedure directly reuses the recorded submission for the supplementary check, without requiring the member to re-execute the sub-task or resubmit the result.

Any conflict arising during the supplementary check is processed according to the preceding conflict-resolution rules, and newly conflict-related sub-tasks are removed from all supplementary checking sets. For a sub-task that remains in a checking set but whose previous submission is no longer included in the valid on-chain records due to conflict resolution, the corresponding validator checks whether the inputs on which its existing result depends are consistent with the task inputs to be used in the current execution. If they are consistent, the validator reuses the existing result; otherwise, it re-executes the sub-task. In either case, it submits the corresponding fingerprint and commitment under the preceding rules. Global consensus is reached and result revealing begins when the valid on-chain records contain fingerprints and commitments from all checking members for every sub-task in their respective $\mathbb{CHECK}_v^r$ and the second global consensus condition is satisfied.

Each supplementary check is assigned a time limit of $\lfloor \frac{T_{\text{commit}}}{K_{\text{check}}} \rfloor$, and the number of supplementary checks in each iteration is bounded by $K_{\text{check}} \in \mathbb{Z}^+$, including the initial check, where T_{commit} is the total timeout of the `VA_COMMIT` state. If a check times out before completion, new checking members are selected from the validators satisfying the first global consensus condition, and new supplementary checking sets are sampled. If a supplementary check has been completed but the second global consensus condition has not yet been satisfied, its checking sets and valid submissions are retained while the procedure waits for the second condition. If all checking attempts are exhausted without completion or the `VA_COMMIT` state reaches its total timeout, `DICE` follows its timeout-handling rules.

Result Revealing (during the `VA_REVEAL` state). In this state, each validator v submits the unrevealed raw VA results $\mathbb{VAR}_v^{\text{reveal}} = \mathbb{VAR}_v \setminus \mathbb{VAR}_v^{\text{commit}}$ for the sub-tasks and input versions corresponding to its valid on-chain submission records (Alg. 1 lines 24–25). These results are submitted to `OnchainReveal()`, which checks whether all of the following conditions hold: (i) $H\big((\mathbb{VAR}_v^{\text{commit}} \cup \mathbb{VAR}_v^{\text{reveal}}) \parallel v\big) = CC_v$; (ii) $H(\mathbb{VAR}_v^{\text{commit}} \cup \mathbb{VAR}_v^{\text{reveal}}) = FP_{\text{agreed}}$; and (iii) $\mathbb{VAR}_v^{\text{commit}} = \mathbb{VAR}_{\text{agreed}}^{\text{commit}}$ (Alg. 2 lines 27–31). Here, FP_{agreed} and $\mathbb{VAR}_{\text{agreed}}^{\text{commit}}$ jointly constitute the consensus fingerprint of the sub-task and input version to which $\mathbb{VAR}_v^{\text{reveal}}$ pertains. All checks are performed against records from the same iteration, sub-task, and valid input version. These checks bind each commitment to the submitter's identity and prevent the direct reuse of another validator's commitment. The formal upper bound on the probability of fraud through copying another validator's results is provided in Theorem 3 of §4.2. Result revealing ends after all validators have revealed the results corresponding to their valid on-chain submission records (line 31).

3.3.2 Efficiency-Driven Incentive Scheme. At the end of `VA_REVEAL`, our EDI scheme rewards or penalizes workers and validators (Alg. 2, line 31) to incentivize honest behavior and encourage participants to contribute more hardware resources to accelerate customization in `DICE`.

Rather than replacing existing incentive schemes [43, 123], EDI augments them with workload-sensitive coefficients. The coefficient $Final_w$ for each worker w, defined in Eq. 7, increases the rewards of honest workers that contribute higher-quality and larger-scale worker updates. The coefficients $Final_v^{\text{pre}}$ and $Final_v^{\text{post}}$ for each validator v, defined in Eq. 8 and 9, respectively, increase the rewards of honest validators that correctly complete more VA sub-tasks before and after the first controversial sub-task appears. If a worker or validator is detected as malicious, its corresponding workload-sensitive coefficient is set to zero. A coefficient is also set to zero if its normalization denominator in Eq. 7–9 is zero.

$$Final_w = \Phi_w\left(1 + \gamma_{worker} \frac{\sum_{j=0}^{N_{lus}-1} Contri_{w,j}}{\sum_{w' \in \mathbb{W}^r} \sum_{j=0}^{N_{lus}-1} Contri_{w',j}}\right), \quad (7)$$

$$Final_v^{\text{pre}} = \Phi_v\left(1 + \gamma_{validator} \frac{N_{v,t}^{\text{correct}} - N_{v,t}^{\text{post}}}{\sum_{v' \in \mathbb{V}^r} N_{v',t}^{\text{pre}}}\right), \quad (8)$$

$$Final_v^{\text{post}} = \Phi_v \left(1 + \gamma_{validator} \frac{N_{v,t}^{\text{post}}}{\sum_{v' \in \mathbb{V}^r} N_{v',t}^{\text{post}}} \right). \tag{9}$$

Here, Φ_w and Φ_v are the base reward weights of worker w and validator v, respectively, which can be determined by an underlying incentive mechanism [123]; both are set to 1 when no such mechanism is used. $\gamma_{worker}, \gamma_{validator} \in \mathbb{R}^+$ are tunable incentive amplification factors. $Contri_{w,j}$ is worker w's contribution score for the j-th model slice, which can be instantiated using the aggregation weight $\alpha_{w,\mathbb{LUS}_j}$ or the worker-contribution metric of the underlying incentive scheme. $N_{v,t}^{\text{correct}}$ is the number of VA sub-tasks correctly completed by validator v, and $N_{v,t}^{\text{post}}$ is the number correctly completed after the first controversial sub-task appears, where $N_{v,t}^{\text{post}} \leq N_{v,t}^{\text{correct}}$. $N_{v,t}^{\text{pre}}$ is the number of distinct sub-tasks completed by v and confirmed on chain when the first controversial sub-task appears. If no controversial sub-task appears in the current round, we set $N_{v,t}^{\text{pre}} = N_{v,t}^{\text{correct}}$. The numbers of correctly completed sub-tasks and the corresponding rewards are determined from each validator's submissions for the input versions associated with the final consensus results. For repeated submissions of the same result for the same sub-task and input version, the earliest on-chain confirmation determines its completion time. The complete on-chain submission history is retained for commitment checks and subsequent accountability and penalties.

The rewards R_w and R_v for workers and validators are given by Eq. 10 and 12, respectively, where WR and VR denote the total worker and validator reward budgets for one round, and Eq. 11 defines the normalization bases used to distribute validator rewards. When a controversial sub-task appears, $Final_{all}^{\text{pre}}$ is computed using the $N_{v,t}^{\text{pre}}$ values fixed when the first controversial sub-task appears. If $\sum_{v \in \mathbb{V}^r} N_{v,t}^{\text{pre}} = 0$, we set $Final_{all}^{\text{pre}} = 0$. Any undistributed reward remains in the task smart contract. The above designs provide participants with additional incentives to improve system efficiency while maintaining compatibility with existing incentive schemes.

$$R_w = \frac{Final_w}{\delta + \sum_{w' \in \mathbb{W}^r} Final_{w'}} \cdot WR, \tag{10}$$

$$\begin{aligned} Final_{all}^{\text{pre}} &= \sum_{v \in \mathbb{V}^r} \Phi_v \left(1 + \gamma_{validator} \frac{N_{v,t}^{\text{pre}}}{\sum_{v' \in \mathbb{V}^r} N_{v',t}^{\text{pre}}} \right), \\ Final_{all}^{\text{post}} &= \sum_{v \in \mathbb{V}^r} Final_v^{\text{post}}. \end{aligned} \tag{11}$$

$$R_v = \left[\frac{Final_v^{\text{pre}}}{\delta + Final_{all}^{\text{pre}}} \left(1 - \frac{N_{CF}}{N_{VA}} \right) + \frac{Final_v^{\text{post}}}{\delta + Final_{all}^{\text{post}}} \frac{N_{CF}}{N_{VA}} \right] \cdot VR, \tag{12}$$

Although the above design encourages active participation, it may also incentivize validators to deliberately trigger conflicts, thereby gaining additional time to complete more sub-tasks and earn higher rewards at the cost of increased consensus latency. To suppress this behavior, EDI uses the committee-confirmed consensus results of controversial sub-tasks and the complete on-chain submission history to identify and penalize submissions that deliberately trigger conflicts. A submission is deemed invalid if its declared input version matches the input version associated with the consensus result but its result fingerprint differs from the consensus fingerprint. The corresponding validator is penalized by burning part of its stake. Eq. 13 gives the amount burned for each invalid submission in a round, where $\epsilon_{penalty} \in \mathbb{Z}^+$ is a predefined base penalty.

$$Punish = \epsilon_{penalty} + \frac{N_{CF}}{N_{VA}} \cdot VR. \tag{13}$$

As shown in Eq. 13, $Punish$ exceeds the post-conflict reward budget $\frac{N_{CF}}{N_{VA}} \cdot VR$, which upper-bounds the additional reward that any validator or colluding group can obtain by triggering conflicts. Moreover, $Punish$ increases with the number of conflict-related sub-tasks. Therefore, for any penalized validator and its colluding group, the additional reward obtained by triggering conflicts cannot offset the corresponding penalty, precluding a positive net payoff from such behavior. A sufficiently large penalty thus economically discourages validators from triggering conflicts to prolong consensus and obtain additional rewards.

3.3.3 Discussion. We finally conclude this section by discussing how TCC and EDI achieve a balance between efficiency, scalability, and security. TCC's core insight is that global consensus does not require every validator to verify every update. Through VA task decomposition and selective committee voting, TCC allows hardware-constrained honest validators to contribute by executing subsets of VA sub-tasks rather than the entire VA task. This shifts the system scalability bottleneck from individual capabilities to the collective power of the validator set, enabling the system to scale to more workers (for larger volumes of private data to be used to improve customized model performance) as the validator scale grows rather than being limited by individual constraints. However, a standalone TCC might degenerate if malicious validators trigger excessive conflicts. EDI tackles this by burning the stake of each malicious validator whose submissions are identified as deliberately triggering conflicts. By setting the penalty above the additional reward obtainable from such conflicts, EDI discourages rational validators from provoking conflict and thus limits the degradation of TCC's efficiency caused by conflict resolution. In §4, we will further show that, with appropriate settings of N_t, N_c, E_c, and ϵ_c, TCC prevents the per-validator workload required to satisfy the global consensus conditions from growing linearly with the number of workers when EDI effectively deters conflicts, thereby improving participant scalability. The proposed design also bounds the success probabilities of fraud and poisoning attacks, providing formal security guarantees.

4 Theoretical Analysis

This section first analyzes DICE's efficiency and security in §4.1 and §4.2, respectively, then discusses a fundamental trade-off revealed from these analyses in §4.3. In §4.1–§4.3, we assume that all participants are active. §4.4 discusses the impacts of non-responsive participants. Finally, §4.5 discusses DICE's on-chain transaction costs.

4.1 Efficiency

This section analyzes the efficiency of our DICE algorithm in terms of per-worker and per-validator computational complexities in each

learning round. A lower per-participant complexity is preferred, as it enables a wider range of honest participants to contribute meaningfully. Moreover, it helps reduce the overall execution time due to the parallel execution of participants within each role.

For ease of analysis, we suppose that (i) the size of the global model is S_{GM}; and (ii) in DICE, the minimum size of the local model is $\gamma_{LM} \cdot S_{GM}$, and the minimum size of the validation model is $\gamma_{VM} \cdot S_{GM}$ (where $0 < \gamma_{VM} \leq \gamma_{LM} \leq 1$). For comparison, we also analyze the current BCFL paradigm introduced in §1 that requires each worker to train the full global model and each validator to verify all worker updates.

Theorem 1 (Per-Participant Complexity in Current BCFL Paradigm). *In any round r of the current BCFL paradigm, the computational complexity of each honest worker is at least $\Omega(S_{GM})$, and the per-validator computational complexity required to satisfy the consensus conditions is at least $\Omega(|\mathbb{W}^r|S_{GM})$.*

Proof. The current BCFL paradigm requires every worker to train the full global model. Therefore, the computational complexity of each honest worker is at least $\Omega(S_{GM})$. Given that $|\mathbb{W}^r|$ workers submit updates of size S_{GM} in round r, each honest validator must verify all updates and participate in cross-validator consensus to receive rewards. Since the current BCFL paradigm typically evaluates each worker update using a validation model of the same size as the global model, the per-validator computational complexity required to satisfy the consensus conditions is at least $\Omega(|\mathbb{W}^r|S_{GM})$.

We next analyze the per-participant complexity of DICE, focusing on the case where EDI effectively deters conflicts, i.e., $N_{CF} = 0$.

Theorem 2 (Per-Participant Complexity in DICE). *In any round r of DICE, the computational complexity of each honest worker is at least $\Omega(\gamma_{LM}S_{GM})$, and the per-validator computational complexity required to satisfy the consensus conditions is at least $\Omega(N_t\gamma_{VM}S_{GM})$.*

Proof. In DICE, each honest worker trains a compact local model whose size is at least $\gamma_{LM}S_{GM}$. Hence, its computational complexity is at least $\Omega(\gamma_{LM}S_{GM})$. Given the updates submitted by workers in $\mathbb{W}^r$, DICE's consensus algorithm divides the task of verifying and aggregating those updates into N_{VA} sub-tasks. When $N_{CF} = 0$, the global consensus conditions in §3.3.1 require at least N_c validators, each of whom submits results for at least N_t distinct sub-tasks. Honest validators counted toward this requirement execute the corresponding sub-tasks according to the protocol. A VER sub-task evaluates a validation model of size at least $\gamma_{VM}S_{GM}$. An AGG sub-task processes the parameters of the corresponding sub-slice. For a given customization task with a fixed model architecture, the ratio of each nonempty sub-slice's parameter size to its corresponding validation model size has a positive constant lower bound. Thus, either type of sub-task requires $\Omega(\gamma_{VM}S_{GM})$ computation, yielding the per-validator lower bound $\Omega(N_t\gamma_{VM}S_{GM})$ required to satisfy the consensus conditions.

Theorems 1 and 2 characterize the lower bounds on the per-participant computational complexities under the current BCFL paradigm and DICE, reflecting their respective participation requirements. Compared with the current BCFL paradigm, DICE reduces the sizes of models that each participant must accommodate for completing a training or verification operation, thus having a greater potential to enable more honest Web3 participants to contribute to customization. More importantly, on the validator side, existing algorithms typically require each validator to verify all worker updates, while DICE allows validators to reach consensus on the complete VA results without individually executing all VA sub-tasks. By setting N_t to scale sub-linearly with $|\mathbb{W}^r|$ (e.g., $N_t = O(\log|\mathbb{W}^r|)$ or even $N_t = O(1)$), DICE makes the lower bound on per-validator computational complexity scale sub-linearly with the number of workers, in contrast to the linear scaling of this bound under the current BCFL paradigm, thereby lowering the validator participation barrier and improving participant scalability. The choice of N_t will be further discussed later in §4.3.

Discussions. The preceding analysis focuses on per-worker and per-validator computational complexities. Beyond these, two additional efficiency dimensions warrant consideration: the requester-side computation overhead, and the system-level communication and coordination complexities. We discuss each below.

Requester-side Computation Overhead. The model requesters in prior BCFL algorithms [51, 55, 64, 75, 77, 82, 130] are typically not involved in the iterative training and VA process after deploying smart contracts for their customization tasks, since those algorithms do not consider asymmetric hardware conditions across participants. In contrast, DICE is designed for resource-asymmetric scenarios and aims to fully exploit requester-side resources to enable lightweight training and verification for hardware-constrained workers and validators.

The requester in DICE undertakes slice selection and large-to-compact model derivation. During slice selection, the requester computes importance scores for candidate slices with complexity $O(\sum_{g=1}^{G} N_{\text{slice},g} \cdot S_{GM})$, where G ($G = 1$ under U-DPL mode) is the number of worker groups with workers in the same group sharing the same hardware capacity indicator L_w, and $N_{\text{slice},g}$ is the number of candidate slices for group g. During large-to-compact model derivation, the requester generates emulators for both workers and validators with complexity $O(N_{\text{emulator}} \cdot S_{GM})$, where N_{emulator} is the number of emulators (each corresponding to a distinct slice in $\{\mathbb{M}_w^r | w \in \mathbb{W}^r\} \cup \{\mathbb{LUS}_j | j \in [0, N_{lus})\}$).

These requester-side costs are deliberate trade-offs for enabling broader participation in customizing large PMs in Web3. The requesters typically have sufficient hardware resources to perform the above computations, and in return, can obtain high-quality customized models and the associated economic benefits. Moreover, the requester can periodically select slices and generate emulators (as discussed in §3.2.3), reducing computation costs over multiple rounds. Further optimization of requester-side costs via more efficient slice evaluation, model distillation, or caching strategies is a promising direction for future work.

Communication and Coordination Complexities. In addition to computation, communication and coordination complexities also affect the overall efficiency of BCFL systems. DICE coordinates all participants via a single smart contract and stores large-scale data (e.g., individual worker updates, emulators, aggregated updates, global model parameters) in an additional distributed storage system like IPFS, as introduced in §2.1.2. Therefore, the total communication and coordination costs comprise two parts: on-chain interactions and off-chain data transfers.

The on-chain interaction complexity depends on the data size of each submission to the smart contract, the number of submissions each participant makes per round and the number of participants.

DICE controls the first factor by recording only necessary data such as cryptographic hashes and file handles on the blockchain. Therefore, our analysis below focuses on measuring on-chain interaction complexity by the latter two factors. The requester and each worker submit at most once per round. The requester publishes the model-slicing results and the file handles and hashes of the corresponding emulators, while each worker submits the file handle and hash of its worker update. Therefore, the on-chain interaction complexities of the requester and each worker are both $O(1)$. Each validator interacts with the smart contract in two states. During VA_COMMIT, a validator submits fingerprints and commitments for the sub-tasks it executes, requiring at most $O(\kappa_v^r N_{v,t})$ submissions. Here, $N_{v,t}$ is the number of sub-tasks executed by validator v in this state, and κ_v^r is the maximum number of fingerprint-and-commitment submissions made by v for one sub-task in round r, including resubmissions that reuse existing results during conflict resolution. During VA_REVEAL, each validator packs its committed but unrevealed results by sub-task and input version into one submission. Therefore, the per-validator on-chain interaction complexities are $O(\kappa_v^r N_{v,t})$ during VA_COMMIT and $O(1)$ during VA_REVEAL. Overall, the total on-chain interaction complexity per round is $O\left(1 + |\mathbb{W}^r| + \sum_{v\in\mathbb{V}^r}(1+\kappa_v^r N_{v,t})\right)$, where the three terms correspond to the requester, all active workers, and all active validators, respectively.

Notably, different from some BCFL algorithms [9, 55, 82] in which each validator submits only once per round, validators in DICE submit VA results in two states: multiple times during VA_COMMIT and once during VA_REVEAL. This design is a deliberate trade-off: using more frequent but lightweight on-chain submissions in exchange for lower per-validator computational complexity bound and security against fraud attacks (proved in Theorem 3). This trade-off is favorable under the consideration that off-chain computation is the primary barrier for hardware-constrained validators in customizing large PMs. Moreover, validators can batch their commit-phase submissions to further reduce the on-chain interaction overhead in practice.

The off-chain data transfer costs are incurred when participants access the distributed storage system. Ignoring repeated downloads, the requester downloads the latest global model and uploads the emulators in a round when the model slicing or emulator generation is triggered, with data transfer volume $O(S_{GM} + \sum_{k=1}^{N_{\text{emulator}}} S_{EM,k})$, where N_{emulator} is the number of emulators uploaded in that round and $S_{EM,k}$ is the size of the k-th emulator. Let $S_{LM,w}$ denote the size of the local model deployed by worker w, $S_{v,t}$ denote the total size of the models, parameters, and off-chain VA results transferred when validator v executes sub-task t, and $\mathbb{T}_v^r$ the set of sub-tasks processed by v in round r. Each worker downloads the global parameters of its assigned slice and the corresponding emulator and uploads its worker update after local training, resulting in transfer volume $O(S_{LM,w})$. Each validator downloads the emulators and worker updates required by its sub-tasks and uploads the corresponding VA results, resulting in transfer volume $O(\sum_{t\in\mathbb{T}_v^r}\kappa_v^r S_{v,t})$. The total off-chain data transfer volume per round is therefore $O\left(S_{GM} + \sum_{k=1}^{N_{\text{emulator}}} S_{EM,k} + \sum_{w\in\mathbb{W}^r} S_{LM,w} + \sum_{v\in\mathbb{V}^r}\sum_{t\in\mathbb{T}_v^r}\kappa_v^r S_{v,t}\right)$. These large data objects are shared through a cost-effective distributed storage system such as IPFS, reducing on-chain storage and communication costs. In addition, reusing emulators across rounds, as discussed in §3.2.3, can further reduce off-chain communication and coordination costs in the corresponding rounds.

4.2 Security

Similar to the existing BCFL design [57], DICE relies on validators to verify workers according to the prescribed rules, filter invalid worker updates, aggregate qualified updates, and reach consensus on the VA results used to update the global model. Security therefore requires preventing malicious validators from both copying others' VA results to fraudulently claim rewards and causing poisoned VA results to be adopted as consensus results. Theorems 3–6 show that TCC bounds the success probabilities of fraud and poisoning attacks while reducing the validator participation barrier. The probability analyses below focus on individual sub-tasks within a given learning round and rely on the following assumptions.

Assumption 1 (VA Result Unpredictability and Hash Function Conditions). *Given any probability threshold $\epsilon_p \in (0,1)$, there exists a specific design for each VA sub-task's result set and a cryptographic hash function $H(\cdot)$ satisfying $2^{-ent} + 2^{-hlen} < \epsilon_p$, where 2^{-ent} is an upper bound on the probability that a malicious validator obtains another validator v's complete VA result $\mathbb{VAR}_v$ over all computational attempts made before the end of* VA_COMMIT*, with a total computational cost substantially lower than that of executing the prescribed computation, hlen is the output bit length of $H(\cdot)$, and its output is modeled as a uniformly random hlen-bit value for any input not queried by the attacker.*

We treat ϵ_p as a security parameter fixed at each customization task initialization and use $\epsilon_p = 10^{-12}$ as the target upper bound on attack success probabilities in the following analyses.

Designs Supporting Assumption 1. To support Assumption 1, each VA sub-task's complete result set includes computation results disclosed only during VA_REVEAL. For VER sub-tasks, those results include validation outputs; for AGG sub-tasks, they include digests computed over the child updates and aggregated update, including a hash chain following a predefined layer order. Numerical VA results are represented using data types with a large representable range (e.g., uint256) and high numerical precision (e.g., 18 decimal places). Collectively, the above designs aim to make it difficult to reconstruct another validator's complete result set from available information at a substantially lower computational cost than generating it through the prescribed computation. Moreover, DICE adopts SHA-256 as the cryptographic hash function, whose 256-bit output satisfies $2^{-256} < \epsilon_p/2$ for the probability thresholds considered.

Assumption 2 (Validator Population and Stake Conditions). *In any learning round, the validator population and total stake satisfy $|\mathbb{V}_{all}| > 2\left\lceil \log_{1-\frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}}\left(\frac{\epsilon_p}{K_{check}}\right)\right\rceil$ and $E_{total} \gg E_c$. The fraction of the total stake held by honest validators, denoted by h, satisfies $h > \frac{2}{3}$. Let $E_{\min}$ and E_{avg} denote the minimum and average validator stakes, respectively; then $\frac{E_{avg}}{E_{\min}} \le \frac{3}{2}$.*

Designs Supporting Assumption 2. To support Assumption 2, DICE incorporates mechanisms concerning stake requirements, validator population, and economic incentives.

First, regarding stake requirements, DICE admits only validators that stake between $E_{\min}$ and $\frac{3}{2}E_{\min}$ basic units of cryptocurrency as a security deposit for a PM customization task, where $E_{\min} = \max(E_c, \epsilon_{penalty} + VR)$. During task execution, any validator whose stake falls outside the range $[E_{\min}, \frac{3}{2}E_{\min}]$, for example, as a result of penalties, is removed. This design enforces the bounded stake ratio $\frac{E_{avg}}{E_{\min}} \le \frac{3}{2}$, limiting stake disparity among individual validators. It also supports deriving a lower bound on the number of honest validators from the honest stake ratio, thereby controlling the validator population required by Theorem 4.

Second, regarding the validator population, DICE recruits at least $\max\left(100, 3\left\lceil \log_{\left[1-\frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}\right]} \frac{\epsilon_p}{K_{\text{check}}} \right\rceil\right)$ validators for each PM customization task and allows new validators to register during task execution. The above stake requirements also apply to these validators. If a validator registers in round r, its public key and stake weight become effective from round $r+2$ onward. This delayed activation prevents newly registered validators from affecting the secrecy and randomness of committee election and voting-power determination. DICE maintains the active validator set on-chain. This set is initialized with the validators recruited before task execution and updated at the end of each round according to the participation records, stake requirements, and activation rounds of newly registered validators. If the updated set contains fewer validators than the above threshold, DICE pauses the next round and reopens validator registration to replenish the set. As an exception to the above delayed-activation rule, validators registered during this pause and their stake weights become effective immediately and are eligible for committee election in the next resumed round. Once the required validator population is restored, DICE first fixes the public keys and stake weights eligible for committee election. The model requester then releases the election seed, after which algorithm execution resumes. Together, the stake requirements and validator population requirement ensure a sufficiently large total stake. In addition, $E_{total} \gg E_c$ and $\frac{E_{avg}}{E_{min}} \le \frac{3}{2}$ make it more difficult for malicious validators to control more than one-third of the total stake, thus promoting $h > \frac{2}{3}$.

Finally, regarding economic incentives, DICE locks validators' stakes and rewards to encourage honest participation and further promote $h > \frac{2}{3}$. Specifically, suppose that validator v stakes cryptocurrency or earns a positive reward in round r of a PM customization task T. DICE locks these tokens from round r until v completes at least $N_t \cdot \epsilon_{lock}$ new VA sub-tasks after round r, either in task T or in subsequent customization tasks, without being detected as malicious, where $\epsilon_{lock} \in \mathbb{Z}^+$. Under this design, validators, particularly those holding a large share of the total stake, have a stronger incentive to remain honest rather than deviate from the algorithm. Honest validators receive rewards for producing correct VA results. In contrast, deviation may cause the model requester to cease submitting new customization tasks if the resulting customized model fails to meet expectations. Previously staked tokens may consequently remain locked indefinitely without corresponding returns and may even be burned if malicious behavior is detected. These reward and withdrawal constraints increase the economic risk of deviation, thereby encouraging rational validators to participate honestly and helping sustain the condition $h > \frac{2}{3}$ in practice.

Under the above assumptions, Theorem 3 presents DICE's resistance to fraud attacks, while Theorems 4–6 and their corollaries show DICE's resistance to poisoning attacks.

Theorem 3 (Resistance to Fraud Attacks). *Under Assumption 1, given any $\epsilon_p \in (0,1)$, the probability that, for any given sub-task and its input version, a malicious validator v', operating at a total computational cost substantially lower than that of executing the prescribed computation, computes and submits $CC_{v'} = H(\mathbb{VAR}_v \,\|\, v')$ using results copied from another honest validator $v \neq v'$ during* VA_COMMIT *is smaller than ϵ_p, where $\mathbb{VAR}_v$ is the complete VA result produced by v for that input version.*

Proof. Given a VA sub-task and one of its input versions, consider a malicious validator v' operating at a total computational cost substantially lower than that of executing the prescribed computation. Let $\mathcal{A}$ be the event that v' obtains the corresponding complete result $\mathbb{VAR}_v$ over all computational attempts made before the end of VA_COMMIT, and let $\mathcal{S}$ be the event that v' submits the correct commitment. By Assumption 1, $\Pr(\mathcal{A}) \le 2^{-ent}$. If $\mathcal{A}$ does not occur, the attacker needs to guess an unknown *hlen*-bit output. Since the on-chain procedure accepts only one commitment from each validator for a given sub-task and input version, $\Pr(\mathcal{S} \cap \overline{\mathcal{A}}) \le 2^{-hlen}$ under the hash output property in Assumption 1. Therefore, $\Pr(\mathcal{S}) = \Pr(\mathcal{S} \cap \mathcal{A}) + \Pr(\mathcal{S} \cap \overline{\mathcal{A}}) \le 2^{-ent} + 2^{-hlen} < \epsilon_p$.

Since submitting the correct commitment is necessary for copied results to pass the checks during VA_REVEAL, Theorem 3 indicates that the fraud attack for any given sub-task and its input version succeeds with probability smaller than ϵ_p.

Theorem 4 (Coverage of Honest Validator Results on Conflict-Unrelated Sub-Tasks). *Under Assumption 2, given any $\epsilon_p \in (0,1)$ and $N_t \in \mathbb{Z}^+$ with $N_t < N_{VA}$, let $N_c = \left\lceil \frac{|\mathbb{V}_{all}|}{2} + \log_{1-\frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}} \left(\frac{\epsilon_p}{K_{check}}\right) \right\rceil$. Then, for any given sub-task, the probability that global consensus is reached and the sub-task is conflict-unrelated at that time, yet no honest validator has executed it according to the protocol and submitted the corresponding fingerprint and commitment, is smaller than ϵ_p.*

Proof. Given any VA sub-task t, let $\mathcal{F}_t$ denote the event that, when global consensus is reached, t is conflict-unrelated but no honest validator has executed it for its current valid input version and submitted the corresponding fingerprint and commitment. Let $\mathcal{A}_i$ denote the event that the i-th supplementary consensus check is initiated, its conflict-unrelated sub-task sampling domain contains t, and none of its honest checking members samples t. Let $\mathcal{H}_i$ denote the execution history before this sampling, and let $\mathbb{V}_{h,i}$ denote the honest checking members. Under Assumption 2, the number of malicious validators satisfies $N_{\text{malicious}} \le \frac{(1-h)E_{avg}|\mathbb{V}_{all}|}{E_{\min}} < \frac{|\mathbb{V}_{all}|}{2}$. Hence, $|\mathbb{V}_{h,i}| \ge N_c - N_{\text{malicious}} > N_c - \frac{|\mathbb{V}_{all}|}{2}$. Conditioned on $\mathcal{H}_i$, the available checking members and conflict-unrelated sub-task sampling domain have been fixed, and the members sample

independently. Since $N_{VA} \le (2|\mathbb{W}^r|-1)N_{lus}$, we have Eq. 14.

$$\Pr(\mathcal{A}_i \mid \mathcal{H}_i) \le \left(1 - \frac{\left\lceil \frac{N_{VA}-N_{CF}}{N_{VA}} N_t \right\rceil}{N_{VA}-N_{CF}}\right)^{|\mathbb{V}_{h,i}|} < \left(1 - \frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}\right)^{N_c - \frac{|\mathbb{V}_{dI}|}{2}} \le \frac{\epsilon_p}{K_{\text{check}}}. \tag{14}$$

This bound applies whenever the corresponding check is initiated and its conflict-unrelated sub-task sampling domain contains t; otherwise, $\mathcal{A}_i$ does not occur. Averaging over the execution histories therefore gives $\Pr(\mathcal{A}_i) < \frac{\epsilon_p}{K_{\text{check}}}$. Under the rules of `SupplementaryCheck()` introduced in §3.3.1, if $\mathcal{F}_t$ occurs, then no honest member can have sampled t in the supplementary check that ultimately passed. Thus, $\mathcal{F}_t \subseteq \bigcup_{i=1}^{K_{\text{check}}} \mathcal{A}_i$. By the union bound, $\Pr(\mathcal{F}_t) \le \sum_{i=1}^{K_{\text{check}}} \Pr(\mathcal{A}_i) < \epsilon_p$.

Theorem 4 indicates that DICE can provide sufficient coverage of honest validator results on conflict-unrelated sub-tasks, thereby supporting the correctness of consensus results adopted by those sub-tasks. This directly yields the following corollaries.

Corollary 1 (Poisoning a Non-Root Sub-Task Triggers Conflict). *Under the same conditions as Theorem 4, for any given non-root VA sub-task, with probability greater than $1-\epsilon_p$, if global consensus is reached and a malicious validator has submitted a poisoned result for the input version ultimately adopted by that sub-task, then the sub-task has become conflict-related. Accordingly, the probability that global consensus is reached while the sub-task remains conflict-unrelated and adopts the poisoned result is smaller than ϵ_p.*

Corollary 2 (Correctness of Non-Root Conflict-Unrelated Sub-Tasks). *Under the same conditions as Theorem 4, for any given non-root VA sub-task, with probability greater than $1-\epsilon_p$, if global consensus is reached while the sub-task remains conflict-unrelated, its adopted consensus result equals the result computed by honest validators according to the protocol for the same valid input version.*

Then, we analyze the correctness of consensus results adopted by conflict-related and root sub-tasks.

Theorem 5 (Correctness of Controversial Sub-Tasks). *Under Assumption 2, given any $\epsilon_p \in (0,1)$, there exist committee election parameters E_c and ϵ_c such that, for any given VA sub-task, the probability that global consensus is reached, the sub-task is controversial at that time, and a poisoned result is adopted as its consensus result is smaller than ϵ_p. Accordingly, with probability greater than $1-\epsilon_p$, if global consensus is reached and the sub-task is controversial, its consensus result equals the result computed by honest validators according to the protocol for the same valid input version.*

Proof. Given any VA sub-task t, honest committee members execute t and all its descendants according to DICE's voting rules for controversial sub-tasks, compute the result according to the protocol for the same valid input version, and vote for the corresponding fingerprint. Let $T = \lfloor E_c\epsilon_c \rfloor + 1$. For any committee election associated with t, adopting a poisoned result requires at least T malicious votes.

Let e_{bad} and e_{good} denote the numbers of malicious and honest votes in a committee election, respectively. According to DICE's committee selection rules, the expected values of e_{bad} and e_{good} are $\mathbb{E}[e_{\text{bad}}] = (1-h)E_c$ and $\mathbb{E}[e_{\text{good}}] = hE_c$. Choose $1-h < \epsilon_c < h$. Then, by the Chernoff bounds for sums of independent Bernoulli random variables, we have $\Pr(e_{\text{bad}} \ge T) \le \exp\left(-\frac{E_c(\epsilon_c+h-1)^2}{\epsilon_c+1-h}\right)$ and $\Pr(e_{\text{good}} < T) \le \exp\left(-\frac{E_c(h-\epsilon_c)^2}{2h}\right)$. It suffices to choose $E_c > \ln\frac{2K_{\text{election}}}{\epsilon_p} \max\left\{\frac{\epsilon_c+1-h}{(\epsilon_c+h-1)^2}, \frac{2h}{(h-\epsilon_c)^2}\right\}$. Then, both tail probabilities are smaller than $\epsilon_p/K_{\text{election}}$. Since $h > \frac{2}{3}$, setting $\epsilon_c = \frac{1}{2}$ gives $\frac{\epsilon_c+1-h}{(\epsilon_c+h-1)^2} < 30$ and $\frac{2h}{(h-\epsilon_c)^2} < 48$. Hence, one valid choice is $\epsilon_c = \frac{1}{2}$ and $E_c = \left\lfloor 48 \ln\frac{2K_{\text{election}}}{\epsilon_p} \right\rfloor + 1$.

If global consensus is reached while t is controversial and a poisoned result is adopted, the malicious vote count must reach T in at least one of the committee elections associated with t. By a union bound over at most K_{election} elections, the probability of this event is smaller than $K_{\text{election}} \cdot (\epsilon_p/K_{\text{election}}) = \epsilon_p$. Consequently, with probability greater than $1-\epsilon_p$, if global consensus is reached while t is controversial, its adopted consensus result equals the result computed by honest validators according to the protocol for the same valid input version.

From Theorem 5, we can derive the correctness of consensus results adopted by conflict-related but non-controversial sub-tasks. Under the eligibility rules of `Voting()`, an honest committee member for a controversial sub-task must execute that sub-task and all its descendants. If the malicious vote count remains below the adoption threshold, any adopted fingerprint must receive an honest member's vote. The corresponding sub-task and its descendants therefore receive submissions from honest validators. By `CheckConflict()`, any descendant that remains non-controversial at global consensus adopts a result consistent with those submitted by honest validators for the same valid input version. Applying Theorem 5's committee voting analysis with threshold ϵ_p/N_{VA} to the controversial ancestors of a given sub-task, and using the fact that their number is at most N_{VA}, yields the following corollary.

Corollary 3 (Correctness of Conflict-Related but Non-Controversial Sub-Tasks). *Under the same conditions as Theorem 5, given any $\epsilon_p \in (0,1)$, there exist committee election parameters E_c and ϵ_c such that, for any given VA sub-task, with probability greater than $1-\epsilon_p$, if global consensus is reached while the sub-task is conflict-related but not controversial, its adopted consensus result equals the result computed by honest validators according to the protocol for the same valid input version.*

Combining Theorems 4 and 5 with Corollary 3 yields an honest-validator result coverage guarantee for any given sub-task, whether conflict-unrelated or conflict-related.

Corollary 4 (Coverage of Honest Validator Results for Any Sub-task). *Under the same conditions as Theorems 4 and 5, given any $\epsilon_p \in (0,1)$, there exist parameters N_c, E_c, and ϵ_c such that, for any given VA sub-task, with probability greater than $1-\epsilon_p$, if global consensus is reached, at least one honest validator has executed the sub-task according to the protocol and submitted the corresponding fingerprint and commitment.*

Specifically, applying Theorem 4 with threshold $\epsilon_p/2$ bounds the probability that no honest validator submits a result in the conflict-unrelated case by $\epsilon_p/2$. In the conflict-related case, applying the committee voting analysis with threshold $\epsilon_p/(2N_{VA})$ and taking a union bound over the given sub-task and its ancestors bounds the corresponding failure probability by $\epsilon_p/2$.

Theorem 6 (Correctness of Non-Controversial Root Sub-Tasks). *Under Assumption 2, given any $\epsilon_p \in (0,1)$, there exist committee election parameters E_c and ϵ_c such that, for any given root sub-task, the probability that global consensus is reached, the root remains non-controversial, and a poisoned result is adopted as its consensus result is smaller than ϵ_p. Accordingly, with probability greater than $1-\epsilon_p$, if global consensus is reached while the root remains non-controversial, its adopted consensus result follows the protocol for the corresponding valid input version.*

Proof. Given any root sub-task t, let $\mathcal{E}_{t,i}$ denote the event that the malicious vote count reaches $\lfloor E_c\epsilon_c \rfloor + 1$ in the i-th committee election for t; if that election does not occur, neither does $\mathcal{E}_{t,i}$. After satisfying the voting eligibility requirements, each honest validator computes the root result according to the protocol for the corresponding valid input version and votes for its fingerprint. Hence, a poisoned result can be adopted only if the malicious vote count reaches the threshold in the corresponding committee. Let $\mathcal{B}_t$ denote the event that global consensus is reached while t remains non-controversial and adopts a poisoned result. Then, $\mathcal{B}_t \subseteq \bigcup_{i=1}^{K_{\text{election}}} \mathcal{E}_{t,i}$. The parameter setting in Theorem 5, i.e., $\epsilon_c = \frac{1}{2}$ and $E_c = \left\lfloor 48 \ln \frac{2K_{\text{election}}}{\epsilon_p} \right\rfloor + 1$, gives $\Pr(\mathcal{B}_t) \le \sum_{i=1}^{K_{\text{election}}} \Pr(\mathcal{E}_{t,i}) < \epsilon_p$. Consequently, with probability greater than $1-\epsilon_p$, if global consensus is reached while the root remains non-controversial, its adopted consensus result follows the protocol for the corresponding valid input version.

Theorems 4–6 and the preceding corollaries together yield DICE's poisoning resistance guarantee. For any given VA sub-task t, apply Theorem 4 with threshold $\epsilon_p/2$. Moreover, for each relevant sub-task, choose the committee election parameters so that the probability of malicious members' votes exceeding the threshold $E_c\epsilon_c$ in any of its at most K_{election} committee elections is smaller than $\epsilon_p/(2N_{VA})$. Then, if t is a conflict-unrelated non-root sub-task, Theorem 4 and Corollary 2 bound the probability of adopting poisoned results. If t is a controversial or root sub-task, adopting a poisoned result requires malicious votes to exceed the threshold in a corresponding committee, according to Theorems 5–6. If t is conflict-related but not controversial, such an event is required to occur in a controversial ancestor of t, according to Corollary 3. Since the total number of sub-tasks comprising t and its ancestors is at most N_{VA}, a union bound gives a probability smaller than $\epsilon_p/2 + N_{VA}\epsilon_p/(2N_{VA}) = \epsilon_p$ that global consensus is reached and t adopts a poisoned result. Therefore, we have the following corollary.

Corollary 5 (Resistance to Poisoning Attacks). *Under the same conditions as Theorems 4–6, given any $\epsilon_p \in (0,1)$, there exist parameters N_c, E_c, and ϵ_c such that, for any given VA sub-task in any round of* DICE*, with probability greater than $1-\epsilon_p$, if global consensus is reached, the sub-task's adopted consensus result follows the protocol for its corresponding valid input version.*

One valid parameter setting is $\epsilon_c = \frac{1}{2}$, $E_c = \left\lfloor 48 \ln \frac{4K_{\text{election}}N_{VA}}{\epsilon_p} \right\rfloor + 1$, and $N_c = \left\lceil \frac{|\mathbb{V}_{all}|}{2} + \log_{1-\frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}} \left(\frac{\epsilon_p}{2K_{\text{check}}}\right) \right\rceil$, with the validator population and stake requirements configured accordingly to satisfy Assumption 2 at the allocated probability thresholds.

Collectively, the efficiency analysis in §4.1, Theorem 3, and Corollary 5 show that TCC lowers the validator participation barrier while bounding the success probabilities of fraud and poisoning attacks, thereby providing security guarantees for the overall customization process.

4.3 Per-Validator Workload Reduction and Scalability Gains versus System Overhead

Theorems 2 and 4 reveal a trade-off between two desirable but conflicting goals. We now discuss this trade-off and navigate it by setting N_t.

- **Goal 1: Reduce the per-validator workload required by the consensus conditions (favoring smaller N_t as shown in Theorem 2)**. This workload affects: (i) *Individual validator overhead and participation barrier*: Lower workload makes it easier for honest participants to contribute as validators. (ii) *Cross-validator consensus latency*: Since validators work in parallel and pipeline off-chain computation of VA sub-tasks without waiting for each sub-task's on-chain confirmation, this latency is dominated by the local computation time of each validator v, characterized by $N_{v,t}\bar{t}_v$, where $\bar{t}_v$ is its average execution time per sub-task. Reducing the required workload can therefore shorten consensus latency. (iii) *System scalability*: The scalability with respect to both workers and validators can be improved significantly when decoupling the per-validator workload requirement from worker scale by setting N_t to $O(1)$.
- **Goal 2: Reduce the required validator scale (favoring larger N_t, as quantified in Theorem 4)**. Reducing N_t increases the number of validators required for security (N_c). A larger N_c brings three issues: (i) *Higher recruitment difficulty*: In ecosystems with limited participants, recruiting N_c or more validators may become infeasible. (ii) *Increased system-level overhead*: Since the complexities of on-chain interaction and off-chain data transfer in each round are both positively correlated with $|\mathbb{V}^r|$ as analyzed in §4.1 and N_c is configured to satisfy $N_c \le |\mathbb{V}^r|$, a larger N_c could increase the system coordination and communication overhead. (iii) Potential offset of latency benefit. Recruiting more validators may involve slower participants with larger $\bar{t}_v$, potentially increasing the maximum $N_{v,t} \times \bar{t}_v$ among all validators and thereby offsetting some latency gains from reducing N_t.

The trade-off between these two goals reflects a common pattern in distributed systems: reducing per-node workload to improve individual efficiency and support larger-scale lower-barrier participation often incurs higher system-level burden [16, 35]. Prior BCFL systems did not encounter this trade-off since they retain the requirement that each validator process all worker updates [51, 55, 64, 75, 77, 82, 130], leaving a critical gap: how to scale customization to hundreds or more workers without assuming all validators possess sufficient hardware for processing worker updates.

DICE navigates this trade-off by using N_t as a tuning knob between the above goals. Fig. 5(left) simulates the effects of varying N_t under fixed worker scale. Fig. 5(right) depicts the growth of N_c with increasing workers under different N_t. For most scenarios, we recommend $N_t = \lceil \log_2 |\mathbb{W}^r| \rceil$ as a balanced choice, which achieves: (i) substantially lower per-validator complexity bound than the current BCFL paradigm (from linear to *sub-linear* in $|\mathbb{W}^r|$), reduced

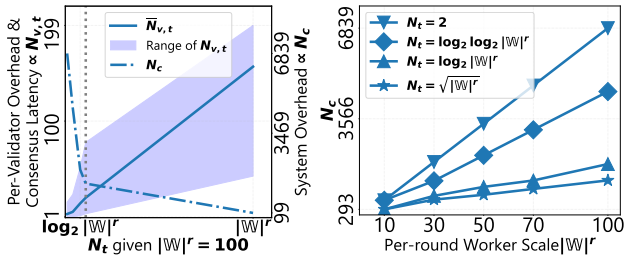


Figure 5: Effects of varying N_t, given $\epsilon_p = 10^{-12}$, $N_{lus} = 1$, $K_{\text{check}} = 1$, $|\mathbb{V}_{all}| = \max(100, 3\lceil \log_{[1-\frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}]} \frac{\epsilon_p}{K_{\text{check}}} \rceil)$, and $N_c = \left\lceil \frac{|\mathbb{V}_{all}|}{2} + \log_{1-\frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}} \left(\frac{\epsilon_p}{K_{\text{check}}} \right) \right\rceil$.

participation barrier, and improved scalability; (ii) *moderate* growth rate on required validator scale and *manageable* system overhead; (iii) *provable* security. DICE offers users the flexibility to adapt N_t to actual deployment constraints such as available participant scale, tolerable system overhead, security requirements, and individual resource budgets.

4.4 Impact of Non-responsive Participants

Below, we discuss the impact of non-responsive participants.

4.4.1 Impacts on Liveness. The customization task may stall in TRAINING, VA_COMMIT, or VA_REVEAL state, if too many participants become non-responsive such that the number of active workers or validators falls below the required thresholds. DICE inherits a non-responsive participant handling scheme from prior work [57], which sets dynamically adjusted timeouts for those states, to prevent deadlock (see §2.3.3). As long as an honest participant remains active and can invoke the task's smart contract, DICE will leave a stalled state rather than wait indefinitely.

4.4.2 Impacts on Efficiency and Model Utility. DICE's overall execution time could increase when non-responsive participants slow down the update collection or the consensus process, or could decrease due to early termination invoked by the non-responsive participant handling scheme. DICE's model performance may degrade if early termination leaves the global model insufficiently trained. Since DICE inherits the non-responsive participant handling scheme from prior work [57], these impacts are similar to those analyzed in that work, and we refer readers to that work for a detailed experimental evaluation of these impacts.

We note that non-responsive participants do not affect the per-participant computational complexity bounds derived in §4.1. These bounds measure individual-level efficiency performance, determined by resource constraint parameters (e.g., L_w, L_v) reported when participants join the customization task or model scale (e.g., S_{GM}) specified by the model requester.

4.4.3 Impacts on Scalability. The greater the number of non-responsive participants, the smaller the pool of available workers and validators, making it harder to meet the required participant scale. DICE mitigates this impact from two directions. On the one hand, DICE's consensus protocol is designed to operate in permissionless blockchain environments, where any device with Internet access can join as a worker or validator without prior authorization. This design choice maximizes the potential participant pool and inherently improves scalability by allowing a larger and more diverse set of participants, thereby increasing resilience to non-responsive participants. On the other hand, as discussed in §4.3, the tunable parameter N_t allows the system to adapt to the actual available participant scale, further mitigating non-responsive participant impacts.

4.4.4 Impacts on Security. As established in §4.2, DICE's security relies on Assumptions 1–2, including the VA result unpredictability and hash function conditions and validator population and stake conditions. Non-responsive honest validators have trivial impact on the VA result unpredictability and hash function conditions, but may reduce the number of validators available for consensus. DICE therefore pauses and replenishes the active validator set as described in §4.2. When DICE resumes the customization process, the security guarantees continue to apply provided that Assumption 2 holds for the validators eligible for the resumed round.

4.5 On-chain Transaction Cost

The overall economic costs of customizing a pretrained model in a typical BCFL system mainly comprise two parts: off-chain computation costs and on-chain transaction costs. DICE focuses on the former for hardware-constrained workers and validators, reducing their per-participant per-round computation workload requirements through DPL, MSV, TCC, and EDI, as analyzed in §4.1. Although DPL and MSV involve off-chain computation for slice selection and emulator generation, this overhead is borne by the requester, who initiates the task and is typically assumed to possess sufficient hardware resources.

For on-chain transaction costs, we use Ethereum [15] as a representative example to illustrate how DICE controls these costs, since BCFL systems can be deployed on various underlying blockchain platforms with different fee models. Ethereum uses gas as a unit to measure the computational and storage resources required to perform an on-chain procedure. The transaction fee for executing an on-chain procedure is calculated as: $fee = gas_used \times gas_price$, where gas_used is the total amount of gas consumed by the smart contract's EVM instructions and data storage, and gas_price is the market-determined price per unit of gas. Following previous work [57], DICE focuses on controlling gas_used, as gas_price is determined by the market rather than by BCFL algorithm design. DICE reduces gas_used by two key designs. First, storing large-scale data, including global model parameters, worker updates and aggregated updates, in an additional cost-effective system such as IPFS. This makes gas_used largely independent of the model size, avoiding the substantial gas overhead incurred by prior BCFL systems that perform on-chain aggregation of model parameters [2, 108, 118]. Second, applying gas-efficient smart contract coding practices [73, 76], such as using low-cost EVM instructions and batching multiple operations into single transactions, to further reduce the gas costs of contract execution. According to previous work [57], the above designs are effective in reducing the ratio of on-chain gas fees to off-chain computation costs, which is indispensable for engaging

a wider range of honest (but potentially hardware-constrained) participants and customizing an effective model.

5 Experiments

We structure a comprehensive evaluation of DICE into five parts to systematically assess its performance under varying conditions. §5.2 evaluates DICE with *honest but hardware-constrained* participants, against an ideal but infeasible scenario where all participants are honest and hardware-unlimited, establishing DICE's performance baseline. §5.3 compares DICE against the current BCFL paradigm in realistic scenarios involving *both hardware-constrained and malicious* participants, demonstrating DICE's holistic advantage. §5.4 studies DICE's flexibility in integrating with existing efficient customization techniques, which, coupled with adjustable N_t discussed in §4.3, enables DICE to accommodate diverse device constraints and model performance requirements. §5.5 extends the evaluation to larger-scale PMs to demonstrate DICE's generalizability beyond the models used in the main experiments. Finally, §5.6 further studies the contribution of DPL and MSV schemes for effective lightweight training and verification and the necessity of using TCC with EDI for efficient consensus.

5.1 Experimental Settings

Our experiments are conducted on a private Ethereum testnet deployed on our in-house private cloud to avoid the real economic costs introduced by invoking smart contracts and making transactions on the public Ethereum mainnet. We follow prior work [57] to configure the blockchain network and refer readers to that work for more details. Below, we introduce the configuration of customization tasks, participants, the BCFL baselines used for comparison, and evaluation metrics.

Task Settings. Our experiments are conducted on nine customization tasks summarized in Table 3. The vision tasks customize ViT [25] for image classification on three datasets commonly used in federated learning research [1, 7, 60, 70, 81, 102, 120, 121]. The language tasks customize BERT [24, 106] for named entity recognition or relation extraction on six biomedical domain datasets from prior work [33, 49, 79, 80]. The extended experiments in §5.5 use larger-scale models, namely DINOv2 [78] for vision tasks and Qwen2.5 [96] for language tasks. For both vision and language tasks, we select one biomedical dataset to evaluate the performance of fine-tuning the PM on that dataset with a prior further-pretraining step. We use MedMNIST [115, 116], a large-scale collection of standardized biomedical images, and PubMed [33], a large-scale biomedical text dataset, for further-pretraining in vision and language tasks, respectively.

Participant Configuration. We deploy 100 workers and 1173 validators for vision customization tasks, and 50 workers and 678 validators for language customization tasks, along with a model requester who submits all customization tasks. These worker scale settings follow prior work [17, 57, 85, 91]. For language tasks, we use a smaller number of workers due to the higher computational costs of customizing BERT and Qwen2.5 compared to ViT and DINOv2 under our experiments. Given worker scale settings, we compute the required number of validators using $|\mathbb{V}_{all}| = \max(100, 3\lceil \log_{[1-\frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}]} \frac{\epsilon_p}{K_{\text{check}}} \rceil)$ with the $N_t = \lceil \log_2 |\mathbb{W}^r| \rceil$ setting recommended in §4.3, $N_{lus} = 3$ (the maximum N_{lus} in our experiments), $K_{\text{election}} = 3$, $K_{\text{check}} = 1$, and a relaxed security threshold $\epsilon_p = 10^{-2}$ that can reduce the required validator scale to achieve a balance between experimental costs and reproducibility. Using these settings and the analysis of Corollary 5, we choose N_c, E_c, and ϵ_c to satisfy the corresponding probability bounds.

Each participant runs in a dedicated virtual machine hosted in our in-house private cloud. We associate each participant with a unique Ethereum testnet account. Each worker is allocated a subset of training samples from the training datasets of the customization tasks. To simulate the non-IID data distribution prevalent in real-world cases, we allocate sample sizes across workers using a Dirichlet distribution and differentiate label distributions across workers based on the number of unique labels in each dataset. Validators have access to a held-out validation dataset to evaluate worker updates. For datasets that include dedicated validation samples, we use them directly. In cases where such samples are unavailable, we split the test set in half, using one half for validation and the other for final model evaluation.

Unless otherwise stated, participants in each experiment are heterogeneous in hardware resources and potentially malicious.

We simulate hardware constraints by limiting the number of H-layers deployable on workers and validators. The number of deployable H-layers on each worker w is set as $L_w = \lfloor \frac{L_G}{2} + \frac{L_G}{2} \cdot \left(1 - \frac{|\mathbb{D}_w| - D_{min}}{D_{max} - D_{min}}\right) \rceil$, where D_{max} and D_{min} are the maximum and minimum sizes of a worker's training dataset, respectively. This simulates a scenario where workers with abundant task-specific data cannot deploy a full model due to hardware limitations. For validators, we simulate hardware heterogeneity by sampling their deployable H-layer counts from a normal distribution: $L_v \sim \mathcal{N}\left(\frac{3L_G}{4}, \frac{L_G^2}{144}\right)$, clipped to the range $[L_G/2, L_G]$. This reflects a common heterogeneous scenario where some validators can deploy nearly full models, while others are constrained and even unable to deploy a worker's complete update when $L_{vm} < L_{wm}$. All L_w and L_v values are rounded to the nearest integer. Table 4 takes Qwen2.5-0.5B, the largest PM used in our experiments, as an example to illustrate the mapping from the total number of H-layers in a pretrained model to actual GPU memory overhead and hardware requirements. This table indicates that configuring deployable H-layers provides a controllable and reproducible proxy scheme to simulate real-world hardware constraints and heterogeneity.

Regarding adversarial settings, we designate 20% of workers and 20% of validators as malicious, with the remaining participants being honest. An exception is the experiment in §5.2, where we configure all participants as honest to establish DICE's performance baseline. Each malicious participant randomly chooses between poisoning and fraud in each round. For any malicious worker, if it chooses to launch fraud attacks, it copies an honest worker's historical update to illegitimately claim rewards. Otherwise, it submits updates trained on mislabeled examples for poisoning attacks [6, 11]. For any malicious validator, if it chooses to launch fraud attacks, it copies then submits another validator's fingerprint and commitment and later submits the copied raw result. Otherwise, it submits poisoned VA results, generated by assigning random

Table 3: Task settings. Tasks marked with ‡ undergo further pretraining before fine-tuning; the others directly fine-tune the PMs.

Settings	Vision Tasks	Language Tasks
Type	Image Classification	NER and RE
Model	ViT-base (default), DINOv2-large	BERT-base (default), Qwen2.5-0.5B
Dataset	FedISIC [1]‡, CIFAR100 [90], DermaMNIST [115, 116]	2018n2c2 [49]‡, BC5-disease (BC5-d) [49], JNLPBA [47], GAD [49], DDI [37], ChemProt [94]
Data Partition	FedISIC: Dirichlet(1.25); all others: Dirichlet(2.5)	
Further Pretraining	MedMNIST (supervised image classification)	PubMed (self-supervised language modeling)
Workers / Validators (N_c)	100 / 1,173 (1,036)	50 / 678 (599)
Total Rounds	100	100
Local Epochs / Batch Size	3 / 50	3 / 64
Learning Rate (LR) Scheduler / Optimizer	Cosine / AdamW	Linear / AdamW
LR	Dynamically tuned at runtime over $\{7\times10^{-5}, 1\times10^{-4}, 3\times10^{-4}, 5\times10^{-4}\}$	
Emulator Distillation Epochs / LR	5 / 1×10^{-4}	
Committee Election / Incentive Parameters	$E_c = 891$, $\epsilon_c = 0.472$, $\gamma_{worker} = 1$, $\gamma_{validator} = 1$, $\epsilon_{penalty} = 10$(×0.01ETH)	
Worker / Validator Reward Budgets	100 (×0.01 ETH) / 100 (×0.01 ETH)	

Table 4: Worker and validator memory overhead (MC_w, MC_v in GB) for model training and multi-update ($|\mathbb{W}^r|$-update) verification when customizing Qwen2.5-0.5B. S_{LM}/S_{VM} denotes local/validation model size in GB. Measurement environment follows Table 1. Hardware Ref. denotes the reference of required hardware.

L_w/L_v	S_{LM}/S_{VM}	MC_w	Hardware Ref.	MC_v	Hardware Ref.
24	1.8	16.4	RTX 4090 24GB ↑	122.2	2× H100 80GB ↑
20	1.6	13.8	RX 9070 XT 16GB ↑	108.3	2 × H100 80GB ↑
16	1.4	11.2	RTX 5070 12GB ↑	94.5	RTX Pro 6000 96GB ↑
12	1.2	8.6	ARC B570 10GB ↑	80.6	2×RTX A6000 48GB ↑

scores above the discard threshold to each malicious worker's update. Unless otherwise stated, all validators stake the same amount of security deposit ($E_{\min}\times0.01$ETH) in each experimental task, and stake-burning penalties are disabled to keep the stake distribution fixed. Therefore, the fraction of total stake controlled by malicious validators equals their share of the total validator count.

BCFL baseline Configuration. We implement the current BCFL paradigm introduced in §1 based on a representative system, namely Refiner [57, 128]. This choice is justified for two reasons. First, Refiner shares the same participant resource requirements as mainstream BCFL algorithms [23, 28, 55, 86, 107], namely requiring each worker to train a full model and each validator to fully verify every worker's update. This allows Refiner to faithfully represent the computational burden of the current BCFL paradigm. Second, Refiner is designed under the same threat model as DICE, addressing both poisoning and fraud attacks. In contrast, many BCFL algorithms [19, 23, 28, 45, 55, 59, 69, 84, 86, 107] neglect fraud attacks that have a negligible impact on model performance, making them unsuitable for a direct comparison under our broader threat model. Owing to the above characteristics, Refiner enables a fair and meaningful comparison of the current BCFL paradigm with DICE.

Our experiments also implement several enhanced variants of the current BCFL paradigm powered by four efficient customization techniques. (i) *LoRA* [41], the most popular PEFT method adopted in federated learning research [93, 111, 129], which exemplifies the family of methods that insert small trainable modules while freezing the backbone of the original pretrained model. (ii) *BitFit* [10], a PEFT approach that does not alter the model architecture but only tunes a subset of selected parameters. Together with LoRA, BitFit covers two fundamentally different types of PEFT techniques discussed in §6 without redundancy. (iii) *Prune* [42, 61, 110] and (iv) *KD* [38, 44, 88, 109], mainstream model compression techniques applicable to both model training and inference scenarios. For the KD-enhanced variants, we use DeiT [98] as the compressed version of ViT and DistilBERT [88] as the compressed version of BERT, respectively. In all baselines, the embedding layers of PMs are frozen.

Evaluation Metrics. The evaluation metrics used in our experiments include (i) Participant Workload Metrics: S_{LM} (GB), the minimum size of local models deployed on the workers, $|\theta_w^r|$ (GB), the maximum size of parameters to be trained on the minimum local model, S_{VM} (GB), the minimum size of validation models deployed on the validators, N_{VM}, the minimum number of validation models that a validator loads during the complete consensus process, among the honest validators contributing to the global consensus conditions. Based on empirical results from previous work [31, 39, 48, 72, 104] and Table 4, S_{LM} and $|\theta_w^r|$ jointly characterize the worker's model-processing workload (including forward/backward computation, activation and gradient memory, and communication), while S_{VM} and N_{VM} characterize the minimum model-deployment requirement and the observed validation workload among honest validators contributing to consensus, and their product is a proxy for cumulative validator model-processing workload. These metrics enable consistent comparisons across different hardware platforms and thus are selected over hardware-dependent measurements such as peak GPU memory or actual execution time to enhance reproducibility. (ii) Model Effectiveness Metrics: *ACC* and *F*1, the accuracy and F1 score of the customized global model on downstream tasks. (iii) System Security Metrics: *RMW* and *RMV*, the proportion of total rewards successfully claimed by malicious workers and validators, respectively.

Below, we run each experiment three times with different random seeds and report the average of the results of the three runs.

Table 5: Results of handling constrained workers (upper half) and constrained validators (lower half). Top-1 and top-2 results are marked in bold and underlined, respectively.

Methods	$\mathbb{W}_{all}$	$S_{LM}\downarrow$	$\lvert\theta_w^r\rvert\downarrow$	ACC ↑			$S_{LM}\downarrow$	$\lvert\theta_w^r\rvert\downarrow$	F1 ↑			$S_{LM}\downarrow$	$\lvert\theta_w^r\rvert\downarrow$	F1 ↑		
		Vision Tasks		FedISIC	CIFAR100	DermaMNIST	NER Tasks		n2c2	JNLPBA	BC5-d	RE Tasks		DDI	GAD	ChemProt
BCFL		100.0%	100.0%	79.5%	94.0%	82.9%	100.0%	100.0%	90.7%	98.5%	89.5%	100.0%	100.0%	44.3%	92.7%	13.8%
+LoRA	✗	101.6%	1.7%	75.7%	91.5%	78.3%	101.2%	1.6%	86.4%	93.5%	85.2%	101.2%	2.2%	43.8%	87.4%	13.1%
+BitFit		100.0%	**0.2%**	71.5%	94.8%	75.2%	100.0%	**0.1%**	81.6%	59.3%	84.5%	100.0%	**0.8%**	42.2%	76.3%	23.0%
+Prune	✓	50.4%	50.0%	95.9%	90.9%	99.6%	**60.9%**	50.0%	97.4%	98.2%	98.5%	**61.1%**	50.4%	93.3%	99.6%	78.8%
+KD		**3.3%**	3.1%	91.7%	80.3%	91.2%			97.1%	99.0%	**99.7%**			92.8%	**99.9%**	97.6%
DICE	✓	50.4%	41.6%	**98.5%**	**98.5%**	**99.8%**	**60.9%**	41.7%	**98.5%**	**99.2%**	98.8%	**61.1%**	42.1%	**96.6%**	99.7%	**99.5%**

Methods	$\mathbb{V}_{all}$	$S_{VM}\downarrow$	$N_{VM}\downarrow$	ACC ↑			$S_{VM}\downarrow$	$N_{VM}\downarrow$	F1 ↑			$S_{VM}\downarrow$	$N_{VM}\downarrow$	F1 ↑		
		Vision Tasks		FedISIC	CIFAR100	DermaMNIST	NER Tasks		n2c2	JNLPBA	BC5-d	RE Tasks		DDI	GAD	ChemProt
BCFL				**100.0%**	**100.0%**	**100.0%**			**100.0%**	**100.0%**	**100.0%**			**100.0%**	**100.0%**	**100.0%**
+LoRA	✗	100.0%		97.2%	98.4%	97.5%	100.0%		96.3%	96.0%	97.2%	100.0%		96.9%	97.3%	94.9%
+BitFit			100.0%	89.9%	92.0%	90.7%		100.0%	91.0%	93.3%	87.5%		100.0%	90.1%	92.3%	89.7%
+Prune	✓	50.4%		95.9%	90.9%	99.6%	**60.9%**		97.4%	98.2%	98.5%	**61.1%**		93.3%	99.6%	78.8%
+KD		**3.3%**		91.7%	80.3%	91.2%			97.1%	99.0%	99.7%			92.8%	99.9%	97.6%
DICE	✓	50.4%	**3.5%**	98.1%	99.4%	99.8%	**60.9%**	**6.0%**	98.1%	99.2%	98.8%	**61.1%**	**6.0%**	97.6%	98.2%	98.7%

Participant workload and model effectiveness metrics are normalized by their values in **an ideal scenario**, where all participants are honest and unconstrained: each worker trains the full model, and each validator executes the complete VA tasks. *RMW* and *RMV* are reported directly as reward fractions and are not normalized by their zero values in the ideal scenario.

5.2 Efficiency Performance

We first evaluate the efficiency of DICE with honest yet hardware-constrained participants against the ideal scenario without constrained participants. To isolate the impact of each participant role's constraints, we conduct two controlled experiments: one with only the workers constrained and another with only the validators constrained. Table 5 reports results of the two experiments and reveals the following findings:

(i) Compared to the current BCFL paradigm, DICE reduces worker model size (S_{LM}) by 40–50% and tunable parameters ($|\theta_w^r|$) by nearly 60%, enabling a substantial reduction in worker-side computational and memory costs. It also reduces validator-side model scale (S_{VM}) by 40–50% and the minimum number of validation models that an honest validator must load to satisfy the global consensus conditions (N_{VM}) by an order of magnitude. Consequently, the product $S_{VM} \times N_{VM}$, which represents validator model-processing workload, becomes substantially smaller. Additionally, DICE maintains 96.6%–99.8% of ideal performance across tasks under both worker-constrained and validator-constrained experiments.

(ii) BCFL and its PEFT-enhanced variants (+LoRA, +BitFit) sacrifice over 10% of ideal performance in most tasks of worker-constrained experiments, as they cannot facilitate workers who are unable to deploy a full model to contribute their task-specific data. Under validator-constrained experiments that remove worker constraints to enable usage of all data, these baselines achieve better ACC/F1. However, these baselines require not only full model deployment and forward computation but also multi-update verification, excluding honest but hardware-constrained participants from engaging as validators for more robust system security.

(iii) Compression-enhanced variants (+Prune, +KD), though effective at enabling constrained participants to engage in customization, suffer 4–20% ACC/F1 drops on complex tasks (e.g., FedISIC, CIFAR100, DDI). The root cause is that the final models customized by these baselines have significantly fewer parameters than the original PMs, which limits their representational capacity for tasks with complex data distributions.

In summary, DICE reduces costs on both workers and validators, lowering the participation barrier while achieving near-ideal model performance. On the worker side, DICE addresses the inherent limitations of compared baselines by DPL. DPL eliminates the full model deployment requirement on constrained participants without compromising model structure on requesters, and adjusts the training pattern dynamically, thus yielding reliable lightweight training across diverse tasks. On the validator side, DICE's efficiency gain is more pronounced. Using MSV, TCC, and EDI, DICE not only reduces per-update verification costs but also resolves a critical scalability bottleneck that prior work cannot address.

5.3 Comparison with Current BCFL Paradigm

We then compare DICE with the current BCFL paradigm and its enhanced variants under a realistic scenario that concurrently incorporates hardware-constrained and malicious participants.

Fig. 6 shows how the performance of PMs customized by different baselines varies as participant costs accumulate. DICE achieves a superior performance frontier, attaining near-ideal ACC/F1 at significantly lower costs than BCFL and its PEFT-enhanced variants, while avoiding the apparent performance degradation observed in compression-enhanced variants on complex tasks (e.g., +Prune on CIFAR100, DDI, ChemProt; +KD on FedISIC, GAD).

Fig. 7 zooms in on the results to examine how DICE addresses the limitation of baselines. The first sub-figure shows that, via DPL and MSV, DICE drastically reduces the size of the model that each worker or validator loads for producing or verifying an update. The second sub-figure shows that, by TCC, DICE significantly reduces the number of validation models that an honest validator must load to satisfy the global consensus conditions. The complementary role of EDI in preserving these efficiency gains is evaluated in

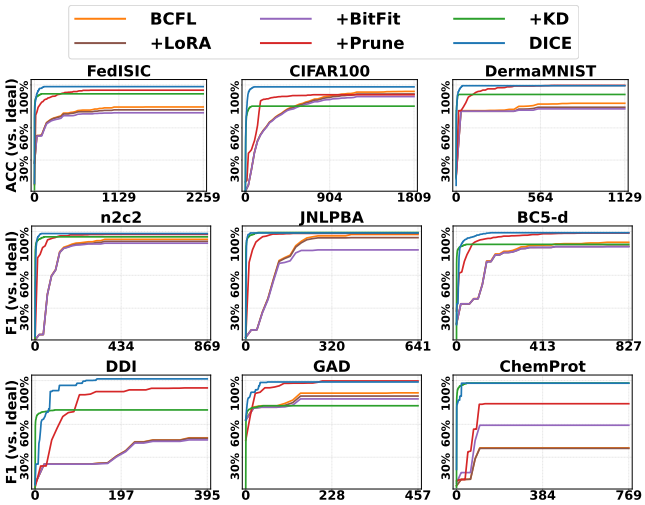


Figure 6: Performance of customized model versus cumulative model-processing workload under different baselines. X-axis: cumulative model-processing workload, measured by accumulated $S_{LM} + N_{VM}S_{VM}$.

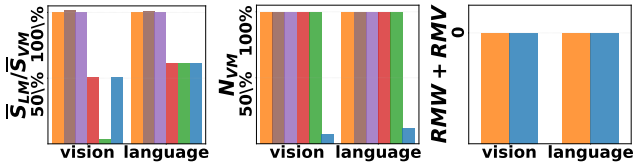


Figure 7: Per-participant workload (S_{LM}, S_{VM}, N_{VM}) and security performance ($RMW + RMV$) under different baselines.

the ablation study in §5.6. Together, the above two sub-figures explain DICE's slower rate of cost accumulation on constrained participants observed in Fig. 6. Finally, the last sub-figure shows that DICE achieves full reward protection, on par with the BCFL paradigm. This indicates that DICE's near-ideal model performance and efficiency gains observed in Fig. 6 do not come at the cost of security.

Collectively, results in Fig. 6–7 not only further corroborate the findings in §5.2, but also confirm that DICE establishes a practical paradigm that simultaneously controls participant expenses, lowers participation barriers, maintains model utility and robust security for customization tasks in real-world Web3 applications.

5.4 Flexibility Performance

Previous experiments show that existing efficient customization techniques, despite having limitations, still offer practical value. For example, PEFT reduces the number of parameters to be trained, while compression achieves competitive accuracy with smaller local/validation models on certain tasks. Therefore, we conduct additional experiments to study DICE's compatibility with these techniques: (i) using PEFT techniques (LoRA, BitFit) together with DPL, and (ii) replacing DPL with compression techniques (Prune, KD). DICE's validator-side designs remain unchanged.

Table 6 shows the results of this experiment and reveals three key findings. First, PEFT techniques can further reduce backward

Table 6: Results of DICE integrated with existing efficient customization techniques on different tasks.

Methods	$S_{LM}\downarrow$	$\lvert\theta_w^r\rvert\downarrow$	$S_{VM}\downarrow$	$N_{VM}\downarrow$	ACC/F1 ↑		
					FedISIC	CIFAR100	DermaMNIST
DICE	50.4%	41.6%	50.4%		98.3%	97.9%	99.1%
+LoRA	51.0%	0.6%	50.4%		94.0%	94.3%	94.6%
+BitFit	50.4%	0.1%	50.4%	3.5%	89.1%	89.2%	89.7%
+Prune	50.4%	50.0%	50.4%		96.1%	90.9%	98.6%
+KD	3.3%	3.1%	3.3%		91.9%	80.1%	91.2%
					n2c2	JNLPBA	BC5-d
DICE	60.9%	41.7%			97.9%	98.7%	98.1%
+LoRA	61.4%	0.6%			94.1%	94.6%	95.9%
+BitFit	60.9%	0.1%	60.9%	6.0%	91.4%	91.7%	93.7%
+Prune	60.9%	50.0%			97.3%	96.9%	97.6%
+KD	60.9%	50.0%			97.1%	98.6%	98.7%
					DDI	GAD	ChemProt
DICE	61.1%	42.1%			96.1%	98.6%	98.3%
+LoRA	61.6%	0.6%			91.1%	94.7%	94.0%
+BitFit	61.1%	0.1%	61.1%	6.0%	90.3%	91.8%	90.8%
+Prune	61.1%	50.4%			92.4%	97.8%	79.4%
+KD	61.1%	50.4%			92.0%	98.9%	97.1%

computation overhead on workers by reducing the update size $|\theta_w^r|$, but they do not reduce the local or validation model sizes S_{LM} and S_{VM}, and thus do not further lower the model-deployment requirements already achieved by DPL and MSV. Second, compression techniques provide alternative paths to simultaneously reduce S_{LM}, $|\theta_w^r|$, and S_{VM}. An extreme reduction is achieved on vision tasks by KD, which uses a much smaller pretrained model, DeiT-Tiny, as a substitute for the requested ViT. Finally, the efficiency gains from these techniques come with trade-offs in model performance, which vary across tasks and methods.

Based on these findings, DICE offers two configuration options. The first option is to use its DPL and MSV schemes with PEFT, extreme compression, or other compatible customization techniques (see §6) to further reduce the overhead of producing and verifying an update, at the cost of potential model utility degradation. The second option is to use the default DPL and MSV to balance efficiency and model performance. Both options work alongside our core innovations, TCC and EDI, which enable BCFL systems to scale to more workers while maintaining security. Together with the adjustable parameter N_t in TCC, these options enable DICE to accommodate diverse device constraints and model performance requirements in practice.

5.5 Customizing DINOv2 and Qwen2.5

This section further evaluates DICE's generalizability by customizing larger-scale pretrained models than those used in the main experiments, including DINOv2 [78] for vision tasks and Qwen2.5 [96] for language tasks. We also report the results of the current BCFL paradigm for reference.

Figs. 8–9 present the results of this experiment. The results consistently reproduce key findings from § 5.3. DICE achieves a superior performance frontier compared to the current BCFL paradigm, attaining higher accuracy/F1 at significantly lower participant workloads across different tasks. Additionally, it continues to effectively reduce the model size loaded by workers and validators,

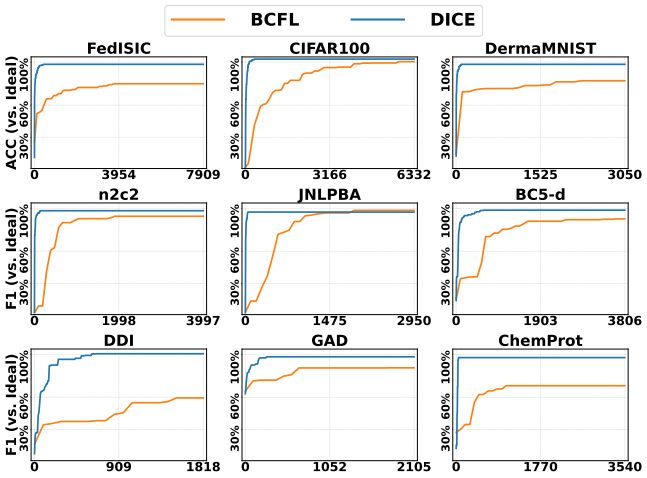


Figure 8: Performance of DINOv2 and Qwen2.5 customized by different baselines. X-axis follows Fig. 6.

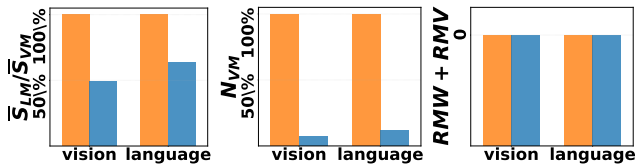


Figure 9: Per-participant workload (S_{LM}/S_{VM}, N_{VM}) and security performance ($RMW + RMV$) under different baselines when customizing DINOv2 and Qwen2.5.

decrease per-validator verification workload, and maintain full reward protection. These results demonstrate that DICE's efficiency and security benefits extend across different model families, supporting its applicability to diverse PM customization tasks in Web3 environments.

5.6 Ablation Study

This section conducts five ablation experiments to demonstrate the contribution of DPL and MSV schemes for effective lightweight training and verification and the necessity of using TCC with EDI for efficient consensus. All experiments use FedISIC (vision), n2c2 (NER), and DDI (RE) as representative tasks.

5.6.1 Effect of DPL's Slice Selection. We first compare DPL's slice selection method against the following alternative methods for determining which H-layers to train in each round: (i) three static methods, namely *T-Only* (training only the top layers), *B-Only* (training only the bottom layers), and *OT* (training both top and bottom layers simultaneously) [110]; (ii) four dynamic methods, namely *DyTB* (progressively training from top to bottom layers), *DyBT* (from bottom to top layers), and *DyOT* (from both ends toward the middle layers) [110], and *Random* (randomly selecting a slice to train in each round).

Table 7 reports the final model performance under different slice selection methods. DPL's method consistently outperforms all alternatives across the three tasks. In contrast, the alternatives exhibit unstable performance across tasks. The static methods (T-Only, B-Only, OT) update only a fixed subset of layers throughout the entire model customization process, failing to explore other potentially informative parts of the model. The dynamic methods (DyTB, DyBT, DyOT) progressively cover more layers over time, yet they follow fixed heuristic traversal orders that do not adapt to the actual data distribution and training progress of model customization tasks. The random method, while able to explore diverse slices across rounds, lacks any principled guidance to prioritize slices with higher utility. DPL's superiority stems from its slice importance scoring, which considers both performance improvement and parameter divergence, enabling the requester to dynamically adjust the training pattern in the actual model customization process.

Table 7: Effect of DPL's importance-score-based slice selection on model performance.

Methods	FedISIC (ACC) ↑	n2c2 (F1) ↑	DDI (F1) ↑
T-Only	73.8%	97.2%	81.6%
B-Only	95.9%	96.1%	86.9%
OT	87.1%	96.9%	76.2%
DyTB	91.7%	97.2%	93.1%
DyBT	96.0%	96.1%	86.9%
DyOT	91.3%	96.9%	88.5%
Random	89.5%	95.8%	83.9%
DPL	98.3%	97.9%	96.1%

Table 8: Effect of emulators (consisting of L_E H-layers) on model performance.

L_E	FedISIC (ACC) ↑	n2c2 (F1) ↑	DDI (F1) ↑
0	55.3%	67.1%	63.8%
1 (Default)	98.3%	97.9%	96.1%
2	98.4%	97.1%	96.3%

5.6.2 Effect of Emulators. We then study the effect of the emulators in approximating the input-output mappings of the global H-layers excluded from the compact model, thereby ensuring effective lightweight training and verification. We vary L_E, the number of H-layers in an emulator, from 0 (no emulator) to 2.

Table 8 shows the resulting model performance. Without an emulator ($L_E = 0$), performance drops significantly. In this setting, any update produced or verified on the compact model is derived from incomplete information. Integrating such an update into the full global model disrupts the original cross-layer semantic information flow of that model. Increasing L_E from 1 to 2 yields limited benefits or even a slight degradation in the final model performance. Moreover, under fixed participant hardware budgets, larger emulators reduce the number of model layers that can be trained or verified in each round. The above results confirm the effectiveness of emulators in approximating input-output mappings of the global H-layers excluded from the compact model, and demonstrate that our default choice $L_E = 1$ balances approximation fidelity and participant-side resource efficiency. Exploring more compact and advanced emulator architectures or data-efficient distillation methods that reduce the requester's overhead represents a promising direction for future work.

Table 9: Effect of MSV's compact-model verification on model performance and security against poisoning attacks from malicious workers.

Methods	ACC/F1 ↑			*RMW* ↓		
	FedISIC	n2c2	DDI	FedISIC	n2c2	DDI
Full-Verify	98.5%	98.5%	96.6%	0.0%	0.0%	0.0%
MSV	98.3%	97.9%	96.1%	0.0%	0.0%	0.0%

Table 10: Effect of dynamic validation threshold on model performance and security against poisoning attacks from malicious workers.

Methods	ACC/F1 ↑			*RMW* ↓		
	FedISIC	n2c2	DDI	FedISIC	n2c2	DDI
Fixed	92.7%	95.4%	92.5%	11.2%	8.7%	9.6%
Dynamic	98.3%	97.9%	96.1%	0.0%	0.0%	0.0%

5.6.3 Effect of MSV's Compact-model-based Verification. Next, we explore the effectiveness of using a compact validation model for verification in MSV by comparing it against full-model verification (Full-Verify), in which validators evaluate each worker's update using the full global model. Table 9 reports the performance of the final model and the fraction of the rewards claimed by malicious workers. MSV's results are comparable to those of Full-Verify across all three tasks, indicating the effectiveness of using a compact validation model as a substitute for the full global model for verification. These results empirically justify MSV's design: inserting an emulator into the compact model captures sufficient information about the global H-layers excluded from that model and thus enables effective verification on that model, and using a dynamic threshold compensates for residual bias. Together, MSV enables hardware-constrained validators to participate while retaining model performance and reward protection comparable to Full-Verify.

5.6.4 Effect of MSV's Dynamic Validation Threshold. We further study the contribution of MSV's dynamic validation threshold by comparing it against a fixed-threshold variant. In this variant, the model requester sets ϵ_j to a constant value at the first round and keeps it unchanged throughout the entire customization process.

Table 10 reports the performance of the final model and the fraction of rewards claimed by malicious workers. With a fixed threshold, the model performance drops below 96% of the ideal ACC/F1, and malicious workers claim about 10% of rewards. The fixed threshold performs poorly because, as the global model improves over rounds, the fixed threshold becomes increasingly lax, making it easier for malicious workers to produce poisoned updates whose validation performance exceeds the threshold. In contrast, the dynamic threshold maintains ACC/F1 above 96% of the ideal and full reward protection across tasks. These results demonstrate that a dynamic validation threshold is crucial for maintaining both model utility and system security, as it tracks the evolving performance of the global model and closes the window that attackers would otherwise exploit.

5.6.5 Necessity of integrating TCC with EDI.. Finally, we examine the role of EDI, particularly its penalty design, in preventing conflict generation and thereby ensuring the efficiency of TCC.

Table 11: Effect of EDI on ensuring TCC's efficiency. The reported results are the average proportion of sub-tasks executed by each honest validator out of all VA sub-tasks in each round.

Methods	All Honest $\overline{N}_{v,t}/N_{VA}$ ↓			20% Malicious $\overline{N}_{v,t}/N_{VA}$ ↓		
	FedISIC	n2c2	DDI	FedISIC	n2c2	DDI
TCC-only	6.3%	10.7%	10.5%	90.5%	88.9%	89.9%
TCC+EDI	6.1%	10.5%	10.6%	6.1%	10.6%	10.5%

Unlike the default experimental settings, this experiment enables EDI's complete reward and penalty mechanism and compares it (TCC+EDI) with a variant (TCC-only) that disables penalties under environments with and without malicious validators. To highlight the effect of EDI, we configure all malicious validators in this experiment to trigger conflicts via poisoning (i.e., submitting VA results that deviate from the consensus) when doing so yields positive net gains, as fraud attacks (submitting copied results) do not generate conflicts.

Table 11 reports the average proportion of VA sub-tasks executed by each honest validator out of all sub-tasks in each round. When all validators are honest, TCC-only and TCC+EDI perform similarly, as no sub-task becomes controversial in this case and thus no extra mandatory workload is incurred for each honest validator. However, when 20% of the validators are malicious, the performance of the two variants diverges sharply. Under TCC-only, the average proportion of VA sub-tasks executed by each honest validator increases by more than 75 percentage points across the three tasks. This is because malicious validators deliberately submit conflicting results to obtain extra time in completing more sub-tasks for more rewards, thereby increasing committee voting procedures for controversial sub-tasks and the prerequisite computation on their descendants. In contrast, TCC+EDI performs similarly under both non-adversarial and adversarial environments, with the slight difference attributable to experimental variance. With EDI, the penalty for submitting invalid results exceeds the maximum potential reward obtainable from triggering conflicts. Consequently, malicious validators have no incentive to provoke controversies, and no additional sub-tasks require committee voting beyond the root sub-tasks. The above results indicate that EDI is important for maintaining TCC's efficiency gains in adversarial environments.

6 Related Work

The work related to DICE mainly falls into three categories: classical federated learning, blockchain-powered federated learning, and efficient customization techniques.

Classical federated learning aims to train a shared global model from data distributed across different workers while protecting data privacy. The first federated learning algorithm, FedAvg [67], and many of its follow-up works [46, 53, 105] proposed an iterative learning scheme that employs a central server to coordinate workers. Data privacy protection is achieved by transmitting only the model parameters, not raw data, between the server and workers. Classical federated learning algorithms are designed for scenarios where at least one participant, e.g., the server, is trusted.

In contrast, our work is designed for Web3 environments where any participant is untrusted.

Blockchain-powered federated learning aims to enable untrusted Web3 participants to contribute to training models through blockchain techniques [55, 64, 82]. Considering the untrusted yet rational and self-interested characteristics of Web3 participants, many studies have proposed (i) well-designed malicious behavior handling schemes [57, 128] that rely on a cross-node consensus algorithm to produce correct global models, and (ii) incentive schemes [43, 57, 123, 128] that incentivize self-interested entities to participate and behave honestly by providing monetary rewards (e.g., cryptocurrency) proportional to their contributions, rather than relying solely on non-monetary incentives such as reputation or service access [43]. The difference between existing BCFL studies and ours is that these prior studies are designed for small models with at most tens of millions of parameters, such as CNN [57, 66], LSTM [57, 66], ResNet [13, 66], while DICE is designed for large pretrained models having at least 100 million parameters, e.g., BERT [24], ViT [25, 109], DINOv2 [78], Qwen [96, 113], LLaMa [99, 127], CLIP [83]. DICE lowers the participant barrier and breaks the scalability bottleneck of the current BCFL paradigm, enabling the system to scale to more workers for better model performance and to more validators for more robust security.

Efficient customization techniques aim to reduce the computational and memory overhead of further pretraining or fine-tuning PMs. These techniques typically follow two paradigms: (i) parameter-efficient fine-tuning [129], such as LoRA [41], Adapter [40], Prompt Tuning [50], and BitFit [10], which retain the pretrained backbone without reducing its size or parameter bit-width; (ii) model compression [112], such as pruning [42, 61, 110], knowledge distillation [38, 88, 109], and quantization [36, 56], which alter the model structure or reduce parameter bit-width. Many studies have integrated these techniques into federated learning to enable hardware-constrained participants to contribute [27]. However, most of them focus on centralized settings, where a powerful trusted server is adopted to verify or aggregate all worker updates [102, 106, 111]. In contrast, DICE is designed for decentralized BCFL systems [23, 55, 57, 59, 84, 107], where verification and aggregation are performed by validators, each of whom is untrusted, potentially hardware-constrained, and required to redundantly execute the same VA tasks to ensure the validity of cross-validator consensus. For existing BCFL systems, the per-validator workload that grows linearly with worker scale becomes the dominant bottleneck hindering them from scaling to a larger number of data holders. Moreover, this cannot be simply resolved by integrating efficient model customization methods.

7 Conclusion

This paper presented DICE, a novel BCFL algorithm designed for efficient and scalable customization of pretrained models in Web3. We systematically analyzed the efficiency and scalability challenges that prevent existing BCFL systems from being directly applied to customize large PMs. To overcome these limitations, we introduced two sets of novel techniques, namely DPL with MSV to support lightweight training and verification of large PMs, and TCC with EDI to resolve the scalability bottleneck that prior BCFL systems cannot address even when enhanced by existing efficient customization methods. We presented a rigorous theoretical analysis of DICE's efficiency, scalability, and security. Extensive experiments on customizing vision and language PMs corroborated our analysis, showing that compared to the current BCFL paradigm, DICE significantly reduces costs for both workers and validators while maintaining competitive model performance and robust security. Moreover, DICE is compatible with existing efficient customization solutions, accommodating diverse device constraints and model performance requirements without compromising security.

Acknowledgments

This work was funded by the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (No. JYB2025XDXM103) and the Leading Talent of Technological Innovation Program (No. 2023R5214) of Zhejiang Province. We also acknowledge anonymous reviewers for their valuable feedback.

References

[1] 2022. FedISIC2019. https://owkin.github.io/FLamby/fed_isic.html
[2] Aftab Akram, Clémentine Gritti, Mohd Hazali Mohamed Halip, Nur Diyana Kamarudin, Marini Mansor, Syarifah Bahiyah Rahayu, and Melek Önen. 2025. Robust blockchain-based federated learning. In *ICISSP 2025, 11th International Conference on Information Systems Security and Privacy, 20-22 February 2025, Porto, Portugal*, Springer (Ed.).
[3] Pierre-Louis Aublin, Sonia Ben Mokhtar, and Vivien Quéma. 2013. RBFT: Redundant byzantine fault tolerance. In *2013 IEEE 33rd international conference on distributed computing systems*. IEEE, 297–306.
[4] PureFed Authors. 2024. PureFed: An Efficient Collaborative and Trustworthy Federated Learning Framework Based on Blockchain Network. *IEEE Access* 12 (2024), 82413–82426.
[5] TBCIM Authors. 2025. TBCIM: Two-Level Blockchain-Aided Edge Resource Allocation Mechanism for Federated Learning Service Market. *IEEE Transactions on Networking* (2025).
[6] Eugene Bagdasaryan, Andreas Veit, Yiqing Hua, Deborah Estrin, and Vitaly Shmatikov. 2020. How To Backdoor Federated Learning. In *AISTATS*, Vol. 108. 2938–2948.
[7] Yu bao Yan, Chun-Mei Feng, Yuexiang Li, Jinheng Xie, Jun Chen, Mohamed Elhoseiny, Ming Hu, Kaishun Wu, and Lei Zhu. 2025. Temporal Model-Based Federated Active Medical Image Classification. In *International Conference on Medical Image Computing and Computer-Assisted Intervention*. https://api.semanticscholar.org/CorpusID:281830599
[8] Marc Bara. 2025. PromptChain: A Decentralized Web3 Architecture for Managing AI Prompts as Digital Assets. *arXiv preprint arXiv:2507.09579* (2025).
[9] Ahmed Ayoub Bellachia, Mouhamed Amine Bouchiha, Yacine Ghamri-Doudane, and Mourad Rabah. 2025. VerifBFL: Leveraging zk-SNARKs for a Verifiable Blockchained Federated Learning. In *NOMS 2025-2025 IEEE Network Operations and Management Symposium*. 01–09.
[10] Elad Ben Zaken, Yoav Goldberg, and Shauli Ravfogel. 2022. BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. 1–9.
[11] Arjun Nitin Bhagoji, Supriyo Chakraborty, Prateek Mittal, and Seraphin Calo. 2019. Analyzing Federated Learning through an Adversarial Lens. In *ICML*, Vol. 97. 634–643.
[12] Avik Bhattacharjee. 2025. Overview of Model Customization. In *A Practical Guide to Generative AI Using Amazon Bedrock: Building, Deploying, and Securing Generative AI Applications*. Springer, 249–281.
[13] Genqing Bian, Wenjing Qu, and Bilin Shao. 2023. Blockchain-based trusted federated learning with pre-trained models for COVID-19 detection. *Electronics* 12, 9 (2023), 2068.
[14] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, S. Buch, Dallas Card, Rodrigo Castellon, Niladri S. Chatterji, Annie S. Chen, Kathleen A. Creel, Jared Davis, Dora Demszky, Chris Donahue, Moussa Koulako Bala Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kawin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren Gillespie, Karan Goel, Noah D. Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E.

Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas F. Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, O. Khattab, Pang Wei Koh, Mark S. Krass, Ranjay Krishna, Rohith Kuditipudi, Ananya Kumar, Faisal Ladhak, Mina Lee, Tony Lee, Jure Leskovec, Isabelle Levent, Xiang Lisa Li, Xuechen Li, Tengyu Ma, Ali Malik, Christopher D. Manning, Suvir Mirchandani, Eric Mitchell, Zanele Munyikwa, Suraj Nair, Avanika Narayan, Deepak Narayanan, Benjamin Newman, Allen Nie, Juan Carlos Niebles, Hamed Nilforoshan, J. F. Nyarko, Giray Ogut, Laurel J. Orr, Isabel Papadimitriou, Joon Sung Park, Chris Piech, Eva Portelance, Christopher Potts, Aditi Raghunathan, Robert Reich, Hongyu Ren, Frieda Rong, Yusuf H. Roohani, Camilo Ruiz, Jack Ryan, Christopher Ré, Dorsa Sadigh, Shiori Sagawa, Keshav Santhanam, Andy Shih, Krishna Parasuram Srinivasan, Alex Tamkin, Rohan Taori, Armin W. Thomas, Florian Tramèr, Rose E. Wang, William Wang, Bohan Wu, Jiajun Wu, Yuhuai Wu, Sang Michael Xie, Michihiro Yasunaga, Jiaxuan You, Matei A. Zaharia, Michael Zhang, Tianyi Zhang, Xikun Zhang, Yuhui Zhang, Lucia Zheng, Kaitlyn Zhou, and Percy Liang. 2021. On the Opportunities and Risks of Foundation Models. *ArXiv* abs/2108.07258 (2021). https://api.semanticscholar.org/CorpusID:237091588

[15] Vitalik Buterin. 2014. A next-generation smart contract and decentralized application platform.

[16] Emanuele Carlini, Patrizio Dazzi, Luca Ferrucci, Jacopo Massa, and Matteo Mordacchini. 2025. Dynamic workload balancing in decentralized edge systems: A marginal cost approach. *Future Generation Computer Systems* (2025), 108167.

[17] Jane Carney, Kushal Upreti, Gaby G Dagher, and Tim Andersen. 2025. FIDELIS: Blockchain-Enabled Protection Against Poisoning Attacks in Federated Learning. *arXiv preprint arXiv:2508.10042* (2025).

[18] Sarthak Chakraborty and Sandip Chakraborty. 2022. Proof of federated training: accountable cross-network model training and inference. In *2022 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, 1–9.

[19] Yuxia Chang, Chen Fang, and Wenzhuo Sun. 2021. A Blockchain-Based Federated Learning Method for Smart Healthcare. *Computational Intelligence and Neuroscience* 2021, 1 (2021), 4376418.

[20] Guorong Chen, Chao Li, Wei Wang, Li Duan, Bin Wang, Zhen Han, and Xiangliang Zhang. 2025. FairReward: Towards Fair Reward Distribution Using Equity Theory in Blockchain-Based Federated Learning. *IEEE Transactions on Dependable and Secure Computing* 22, 2 (2025), 1612–1626.

[21] Leiming Chen, Dehai Zhao, Liping Tao, Kai Wang, Sibo Qiao, Xingjie Zeng, and Chee Wei Tan. 2025. A Credible and Fair Federated Learning Framework Based on Blockchain. *IEEE Transactions on Artificial Intelligence* 6, 2 (2025), 301–316.

[22] Jiao Dai, He Huang, Yufei Wang, Li Duan, Fuqiang Hu, Dantong Yan, Zhichen Liu, and Wei Wang. 2025. Byzantine-resistant model verification for decentralized federated learning in intelligent transportation systems. *Applied Soft Computing* 169 (2025), 111852.

[23] Harsh Bimal Desai, Mustafa Safa Ozdayi, and Murat Kantarcioglu. 2021. Blockfla: Accountable federated learning via hybrid blockchain architecture. In *Proceedings of the eleventh ACM conference on data and application security and privacy*. 101–112.

[24] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *CoRR* abs/1810.04805 (2018). arXiv:1810.04805 http://arxiv.org/abs/1810.04805

[25] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ICLR* (2021).

[26] John R. Douceur. 2002. The Sybil Attack. In *Peer-to-Peer Systems*. Springer Berlin Heidelberg, 251–260.

[27] Tao Fan, Hanlin Gu, Xuemei Cao, Chee Seng Chan, Qian Chen, Yiqiang Chen, Yihui Feng, Yang Gu, Jiaxiang Geng, Bing Luo, et al. 2025. Ten challenging problems in federated foundation models. *IEEE Transactions on Knowledge and Data Engineering* (2025).

[28] Lei Feng, Yiqi Zhao, Shaoyong Guo, Xuesong Qiu, Wenjing Li, and Peng Yu. 2022. BAFL: A Blockchain-Based Asynchronous Federated Learning Framework . *IEEE Trans. Comput.* 71, 05 (2022), 1092–1103.

[29] Ocean Protocol Foundation. 2020. Ocean Protocol: A Decentralized Data Exchange Protocol to Unlock Data for AI. https://oceanprotocol.com/tech-whitepaper.pdf

[30] Wenyu Gao, Gaochao Xu, and Xianqiu Meng. 2025. FedCon: Scalable and Efficient Federated Learning via Contribution-Based Aggregation. *Electronics* 14, 5 (2025).

[31] Stefanos Georgiou, Maria Kechagia, and Federica Sarro. 2022. Green AI: Do Deep Learning Frameworks Have Different Costs? *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)* (2022), 1082–1094. https://api.semanticscholar.org/CorpusID:247614182

[32] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, and N. Zeldovich. 2017. Algorand: Scaling Byzantine Agreements for Cryptocurrencies. In *SOSP*. 51–68.

[33] Yu Gu, Robert Tinn, Hao Cheng, Michael Lucas, Naoto Usuyama, Xiaodong Liu, Tristan Naumann, Jianfeng Gao, and Hoifung Poon. 2021. Domain-Specific Language Model Pretraining for Biomedical Natural Language Processing. *ACM Trans. Comput. Healthcare* 3, 1 (2021).

[34] Ehsan Hallaji, Roozbeh Razavi-Far, Mehrdad Saif, Boyu Wang, and Qiang Yang. 2024. Decentralized federated learning: A survey on security and privacy. *IEEE Transactions on Big Data* 10, 2 (2024), 194–213.

[35] Heiko Hamann and Andreagiovanni Reina. 2021. Scalability in computing and robotics. *IEEE Trans. Comput.* 71, 6 (2021), 1453–1465.

[36] Song Han, Huizi Mao, and William J Dally. 2015. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149* (2015).

[37] María Herrero-Zazo, Isabel Segura-Bedmar, Paloma Martínez, and Thierry Declerck. 2013. The DDI corpus: An annotated corpus with pharmacological substances and drug–drug interactions. *Journal of biomedical informatics* 46, 5 (2013), 914–920.

[38] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531* (2015).

[39] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. 2022. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556* (2022).

[40] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-Efficient Transfer Learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*.

[41] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. LoRA: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.

[42] Lanxiang Hu, Tajana Rosing, and Hao Zhang. 2025. TrimLLM: Progressive Layer Dropping for Domain-Specific LLMs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 667–681.

[43] Cornelius Ihle, Dennis Trautwein, Moritz Schubotz, Norman Meuschke, and Bela Gipp. 2023. Incentive Mechanisms in Peer-to-Peer Networks — A Systematic Literature Review. *Comput. Surveys* 55, 14s (2023).

[44] Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. 2020. Tinybert: Distilling bert for natural language understanding. In *Findings of the association for computational linguistics: EMNLP 2020*. 4163–4174.

[45] Aditya Pribadi Kalapaaking, Ibrahim Khalil, and Xun Yi. 2023. Blockchain-based federated learning with SMPC model verification against poisoning attack for healthcare systems. *IEEE Transactions on Emerging Topics in Computing* 12, 1 (2023), 269–280.

[46] Sai Praneeth Karimireddy, S. Kale, M. Mohri, Sashank J. Reddi, Sebastian U. Stich, and A. T. Suresh. 2020. SCAFFOLD: Stochastic Controlled Averaging for Federated Learning. In *ICML*.

[47] Jin-Dong Kim, Tomoko Ohta, Yoshimasa Tsuruoka, Yuka Tateisi, and Nigel Collier. 2004. Introduction to the Bio-Entity Recognition Task at JNLPBA. In *Proceedings of the International Joint Workshop on Natural Language Processing in Biomedicine and its Applications (NLPBA/BioNLP)*. COLING, Geneva, Switzerland, 73–78.

[48] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems* 5 (2023), 341–353.

[49] Jinhyuk Lee, Wonjin Yoon, Sungdong Kim, Donghyeon Kim, Sunkyu Kim, Chan Ho So, and Jaewoo Kang. 2020. BioBERT: a pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics* 36, 4 (2020), 1234–1240.

[50] Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The Power of Scale for Parameter-Efficient Prompt Tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 3045–3059.

[51] Dun Li, Dezhi Han, Tien-Hsiung Weng, Zibin Zheng, Hongzhi Li, Han Liu, Arcangelo Castiglione, and Kuan-Ching Li. 2022. Blockchain for federated learning toward secure distributed machine learning systems: a systemic survey. *Soft Computing* 26, 9 (2022), 4423–4440.

[52] Kai Li, Zhicai Zhang, and Fang Fu. 2023. An Incentive Mechanism for Consortium Blockchain-based Cross-Silo Federated Learning. In *2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS)*. 451–458.

[53] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. 2020. Federated Optimization in Heterogeneous Networks. In *MLSys*, Vol. 2. 429–450.

[54] Wenyu Li, Chenglin Feng, Lei Zhang, Hao Xu, Bin Cao, and Muhammad Ali Imran. 2020. A scalable multi-layer PBFT consensus for blockchain. *IEEE Transactions on Parallel and Distributed Systems* 32, 5 (2020), 1146–1160.

[55] Yuzheng Li, Chuan Chen, Nan Liu, Huawei Huang, Zibin Zheng, and Qiang Yan. 2021. A Blockchain-Based Decentralized Federated Learning Framework with Committee Consensus. *IEEE Network* 35, 1 (2021), 234–241.

[56] Tailin Liang, John Glossner, Lei Wang, Shaobo Shi, and Xiaotong Zhang. 2021. Pruning and quantization for deep neural network acceleration: A survey. *Neurocomputing* 461 (2021), 370–403.

[57] Hong Lin, Ke Chen, Dawei Jiang, Lidan Shou, and Gang Chen. 2024. Refiner: a reliable and efficient incentive-driven federated learning system powered by blockchain. *The VLDB Journal* 33, 3 (2024), 807–831.

[58] Hong Lin, Shixin Wan, Zhongle Xie, Ke Chen, Meihui Zhang, Lidan Shou, and Gang Chen. 2025. A Comprehensive Study of Shapley Value in Data Analytics. *Proc. VLDB Endow.* 18, 9 (2025), 3077–3092.

[59] Yuan Liu, Zhengpeng Ai, Shuai Sun, Shuangfeng Zhang, Zelei Liu, and Han Yu. 2020. *FedCoin: A Peer-to-Peer Payment System for Federated Learning*. 125–138.

[60] Charles Lu and Jayasheree Kalpathy-Cramer. 2022. Distribution-Free Federated Learning with Conformal Predictions. arXiv:2110.07661

[61] Yao Lu, Hao Cheng, Yujie Fang, Zeyu Wang, Jiaheng Wei, Dongwei Xu, Qi Xuan, Xiaoniu Yang, and Zhaowei Zhu. 2024. Reassessing layer pruning in LLMs: New insights and methods. *arXiv preprint arXiv:2411.15558* (2024).

[62] Yuan Lu, Qiang Tang, and Guiling Wang. 2018. On enabling machine learning tasks atop public blockchains: A crowdsourcing approach. In *2018 IEEE international conference on data mining workshops (ICDMW)*. IEEE, 81–88.

[63] Elizabeth Lui and Jiahao Sun. 2025. Bittensor Protocol: The Bitcoin in Decentralized Artificial Intelligence? A Critical and Empirical Analysis. *arXiv preprint arXiv:2507.02951* (2025).

[64] Chuan Ma, Jun Li, Long Shi, Ming Ding, Taotao Wang, Zhu Han, and H. Vincent Poor. 2022. When Federated Learning Meets Blockchain: A New Distributed Learning Paradigm. *IEEE Computational Intelligence Magazine* 17, 3 (2022), 26–33.

[65] Xuyang Ma, Du Xu, and Katinka Wolter. 2024. Towards blockchain-enabled decentralized and secure federated learning. *Information Sciences* 662 (2024).

[66] T Manoj, Krishnamoorthi Makkithaya, and VG Narendra. 2025. A blockchain-assisted trusted federated learning for smart agriculture. *SN Computer Science* 6, 3 (2025), 221.

[67] H. B. McMahan, Eider Moore, Daniel Ramage, S.C.D. Hampson, and Blaise Agüera y Arcas. 2017. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *AISTATS*, Vol. 54. 1273–1282.

[68] MedSynapse. 2026. MedSynapse Official Website. https://ethglobal.com/showcase/medsynapse-kyw2m

[69] Brandon Alejandro Mosqueda González, Omar Hasan, and Lionel Brunie. 2025. Mitigation of Sybil-based Poisoning Attacks in Permissionless Decentralized Learning. In *2025 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, 1–5. https://hal.science/hal-05002718

[70] Noorain Mukhtiar, Adnan Mahmood, Yipeng Zhou, Jian Yang, Jing Teng, and Quan Z. Sheng. 2024. Federated Learning at the Forefront of Fairness: A Multifaceted Perspective. In *International Joint Conference on Artificial Intelligence*. https://api.semanticscholar.org/CorpusID:281484496

[71] Satoshi Nakamoto. 2008. Bitcoin: A Peer-to-Peer Electronic Cash System.

[72] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. 2021. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*. 1–15.

[73] Keerthi Nelaturu, Sidi Mohamed Beillahi, Fan Long, and Andreas G. Veneris. 2021. Smart Contracts Refinement for Gas Optimization. *BRAINS* (2021), 229–236.

[74] Mind Network. 2026. Product of Mind Network: World AI Health Hub. https://docs.mindnetwork.xyz/minddocs/product/agenticworld/agenticworld-user-guide/world-ai-health-hub

[75] Dinh C Nguyen, Ming Ding, Quoc-Viet Pham, Pubudu N Pathirana, Long Bao Le, Aruna Seneviratne, Jun Li, Dusit Niyato, and H Vincent Poor. 2021. Federated learning meets blockchain in edge computing: Opportunities and challenges. *IEEE Internet of Things Journal* 8, 16 (2021), 12806–12825.

[76] Quang-Thang Nguyen, Bao Son Do, Thi Tam Nguyen, and Ba-Lam Do. 2022. GasSaver: A Tool for Solidity Smart Contract Optimization. *ACM BSCI* (2022).

[77] Weiguang Ning, Yingjuan Zhu, Caixia Song, Hongxia Li, Lihui Zhu, Jinbao Xie, Tianyu Chen, Tong Xu, Xi Xu, and Jiwei Gao. 2024. Blockchain-Based Federated Learning: A Survey and New Perspectives. *Applied Sciences* 14, 20 (2024).

[78] Maxime Oquab, Timothée Darcet, Theo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Russell Howes, Po-Yao Huang, Hu Xu, Vasu Sharma, Shang-Wen Li, Wojciech Galuba, Mike Rabbat, Mido Assran, Nicolas Ballas, Gabriel Synnaeve, Ishan Misra, Herve Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. 2023. DINOv2: Learning Robust Visual Features without Supervision.

[79] Le Peng, Gaoxiang Luo, Sicheng Zhou, Jiandong Chen, Ziyue Xu, Ju Sun, and Rui Zhang. 2024. An in-depth evaluation of federated learning on biomedical natural language processing for information extraction. *npj Digit. Medicine* 7, 1 (2024).

[80] Le Peng, sicheng zhou, jiandong chen, Rui Zhang, Ziyue Xu, and Ju Sun. 2023. A Systematic Evaluation of Federated Learning on Biomedical Natural Language Processing. https://openreview.net/forum?id=pLEQFXACNA

[81] Sara Pieri, Jose Renato Restom, Samuel Horvath, and Hisham Cholakkal. 2023. Handling data heterogeneity via architectural design for federated visual recognition. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*.

[82] Youyang Qu, Md Palash Uddin, Chenquan Gan, Yong Xiang, Longxiang Gao, and John Yearwood. 2022. Blockchain-enabled Federated Learning: A Survey. *ACM Comput. Surv.* 55, 4 (2022), 35 pages.

[83] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *International Conference on Machine Learning*. https://api.semanticscholar.org/CorpusID:231591445

[84] Paritosh Ramanan and Kiyoshi Nakayama. 2020. BAFFLE : Blockchain Based Aggregator Free Federated Learning. In *2020 IEEE International Conference on Blockchain (Blockchain)*. 72–81.

[85] Shuyang Ren, Eunsam Kim, and Choonhwa Lee. 2024. A scalable blockchain-enabled federated learning architecture for edge computing. *Plos one* 19, 8 (2024), e0308991.

[86] Yanli Ren, Mingqi Hu, Zhe Yang, Guorui Feng, and Xinpeng Zhang. 2024. BPFL: Blockchain-based privacy-preserving federated learning against poisoning attack. *Information Sciences* 665 (2024), 120377.

[87] Fahad Saleh. 2021. Blockchain without waste: Proof-of-stake. *The Review of Financial Studies* 34, 3 (2021), 1156–1190.

[88] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108* (2019).

[89] Johannes Schneider, Christian Meske, and Pauline Kuss. 2024. Foundation models: A new paradigm for artificial intelligence. *Business & Information Systems Engineering* 66, 2 (2024), 221–231.

[90] Neha Sharma, Vibhor Jain, and Anju Mishra. 2018. An Analysis Of Convolutional Neural Networks For Image Classification. *Procedia Computer Science* 132 (2018), 377–384. International Conference on Computational Intelligence and Data Science.

[91] Muhammad Shayan, Clement Fung, Chris JM Yoon, and Ivan Beschastnikh. 2020. Biscotti: A blockchain system for private and secure federated learning. *IEEE Transactions on Parallel and Distributed Systems* 32, 7 (2020), 1513–1525.

[92] Meng Shen, Zhehui Tan, Dusit Niyato, Yuzhi Liu, Jiawen Kang, Zehui Xiong, Liehuang Zhu, Wei Wang, and Xuemin Shen. 2024. Artificial Intelligence for Web 3.0: A Comprehensive Survey. *Comput. Surveys* 56, 10 (2024), 1–39.

[93] Zhanming Shen, Tianqi Xu, Hao Wang, Jian Li, and Miao Pan. 2025. pFedGPT: Hierarchically Optimizing L,oRA, Aggregation Weights for Personalized Federated GPT, Models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 4766–4778.

[94] Olivier Taboureau, Sonny Kim Nielsen, Karine Audouze, Nils Weinhold, Daniel Edsgä, rd, Francisco S Roque, Irene Kouskoumvekaki, Alina Bora, Ramona Curpan, t Jensen, Thomas Skø, et al. 2010. ChemProt: a disease chemical biology database. *Nucleic acids research* 39 (2010), D367–D372.

[95] Nurbek Tastan, Samar Fares, Toluwani Aremu, Samuel Horvath, and Karthik Nandakumar. 2024. Redefining contributions: shapley-driven federated learning. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*. Article 554, 9 pages.

[96] Qwen Team. 2024. Qwen2.5: A Party of Foundation Models. https://qwenlm.github.io/blog/qwen2.5/

[97] Youliang Tian, Ta Li, Jinbo Xiong, Md Zakirul Alam Bhuiyan, Jianfeng Ma, and Changgen Peng. 2021. A blockchain-based machine learning framework for edge services in IIoT. *IEEE Transactions on Industrial Informatics* 18, 3 (2021), 1918–1929.

[98] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. 2021. Training data-efficient image transformers & distillation through attention. In *International conference on machine learning*. PMLR, 10347–10357.

[99] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv* (2023). arXiv:2302.13971

[100] Kentaroh Toyoda, Jun Zhao, Allan Neng Sheng Zhang, and P Takis Mathiopoulos. 2020. Blockchain-enabled federated learning with mechanism design. *IEEE Access* 8 (2020), 219744–219756.

[101] Nguyen Truong, Kai Sun, Siyao Wang, Florian Guitton, and YiKe Guo. 2021. Privacy preservation in federated learning: An insightful survey from the GDPR perspective. *Computers & Security* 110 (2021), 102402.

[102] Adam Tupper and Christian Gagn'e. 2025. Personalized Federated Fine-Tuning of Vision Foundation Models for Healthcare. *ArXiv* abs/2510.12741 (2025). https://api.semanticscholar.org/CorpusID:282064942

[103] Muhammad Habib ur Rehman, Ahmed Mukhtar Dirir, Khaled Salah, Ernesto Damiani, and Davor Svetinovic. 2021. TrustFed: A framework for fair and trustworthy cross-device federated learning in IIoT. *IEEE Transactions on Industrial Informatics* 17, 12 (2021), 8485–8494.

[104] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, ukasz Kaiser, Ł, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[105] Hongyi Wang, Mikhail Yurochkin, Yuekai Sun, Dimitris Papailiopoulos, and Yasaman Khazaeni. 2020. Federated Learning with Matched Averaging. In *ICLR*.

[106] Xin'ao Wang, Huan Li, Ke Chen, and Lidan Shou. 2023. FedBFPT: An Efficient Federated Learning Framework for BERT Further Pre-training. In *International Joint Conference on Artificial Intelligence*. https://api.semanticscholar.org/CorpusID:260859911

[107] Jiasi Weng, Jian Weng, Jilian Zhang, Ming Li, Yue Zhang, and Weiqi Luo. 2021. DeepChain: Auditable and Privacy-Preserving Deep Learning with Blockchain-Based Incentive. *IEEE Trans. Dependable Secur. Comput.* 18, 5 (2021), 2438–2455.

[108] Francesc Wilhelmi, Lorenza Giupponi, and Paolo Dini. 2021. Blockchain-enabled server-less federated learning. *arXiv preprint arXiv:2112.07938* (2021), 1–14.

[109] Kan Wu, Jinnian Zhang, Houwen Peng, Mengchen Liu, Bin Xiao, Jianlong Fu, and Lu Yuan. 2022. TinyViT: Fast pretraining distillation for small vision transformers. In *European conference on computer vision*. Springer, 68–85.

[110] Guangxuan Xiao, Ji Lin, and Song Han. 2023. Offsite-tuning: Transfer learning without full model. *arXiv preprint arXiv:2302.04870* (2023).

[111] Honghui Xu, Shiva Shrestha, Wei Chen, Zhiyuan Li, and Zhipeng Cai. 2025. DP-FedLoRA: Privacy-Enhanced Federated Fine-Tuning for On-Device Large Language Models. *arXiv preprint arXiv:2509.09097* (2025).

[112] Mengwei Xu, Dongqi Cai, Wangsong Yin, Shangguang Wang, Xin Jin, and Xuanzhe Liu. 2025. Resource-efficient Algorithms and Systems of Foundation Models: A Survey. *ACM Comput. Surv.* 57, 5, Article 110 (Jan. 2025), 39 pages. https://doi_.org/10.1145/3706418

[113] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zhihao Fan. 2024. Qwen2 Technical Report. *arXiv preprint arXiv:2407.10671* (2024).

[114] Bang Yang, Asif Raza, Yuexian Zou, and Tong Zhang. 2023. Customizing general-purpose foundation models for medical report generation. *arXiv preprint arXiv:2306.05642* (2023).

[115] Jiancheng Yang, Rui Shi, and Bingbing Ni. 2021. MedMNIST Classification Decathlon: A Lightweight AutoML Benchmark for Medical Image Analysis. In *IEEE 18th International Symposium on Biomedical Imaging (ISBI)*. 191–195.

[116] Jiancheng Yang, Rui Shi, Donglai Wei, Zequan Liu, Lin Zhao, Bilian Ke, Hanspeter Pfister, and Bingbing Ni. 2023. MedMNIST v2-A large-scale lightweight benchmark for 2D and 3D biomedical image classification. *Scientific Data* 10, 1 (2023), 41.

[117] Qing Yang, Wei Xu, Taotao Wang, Hao Wang, Xiaoxiao Wu, Bin Cao, and Shengli Zhang. 2024. Blockchain-Based Decentralized Federated Learning With On-Chain Model Aggregation and Incentive Mechanism for Industrial IoT. *IEEE Open Journal of the Communications Society* 5 (2024), 6420–6429.

[118] Qing Yang, Wei Xu, Taotao Wang, Hao Wang, Xiaoxiao Wu, Bin Cao, and Shengli Zhang. 2024. Blockchain-Based Decentralized Federated Learning With On-Chain Model Aggregation and Incentive Mechanism for Industrial IoT. *IEEE Open Journal of the Communications Society* 5 (2024), 6420–6429.

[119] Xiaohui Yang and Tianchang Li. 2024. A Blockchain-based federated learning framework for secure aggregation and fair incentives. *Connection Science* 36 (2024). https://api.semanticscholar.org/CorpusID:267970430

[120] Mang Ye, Wenke Huang, Zekun Shi, He Li, and Bo Du. 2025. Revisiting federated learning with label skew: an over-confidence perspective. *Science China Information Sciences* 68 (2025). https://api.semanticscholar.org/CorpusID:279980636

[121] Musa Yenilmez and Ilhan Aydin. 2025. Particle Swarm Optimized Federated Learning for Efficient Classification of Medical Images. In *2025 15th International Conference on Advanced Computer Information Technologies (ACIT)*. 868–872.

[122] Guangsheng Yu, Xu Wang, Caijun Sun, Qin Wang, Ping Yu, Wei Ni, and Ren Ping Liu. 2023. IronForge: An open, secure, fair, decentralized federated learning. *IEEE Transactions on Neural Networks and Learning Systems* 36, 1 (2023), 354–368.

[123] Yufeng Zhan, Jie Zhang, Zicong Hong, Leijie Wu, Peng Li, and Song Guo. 2022. A Survey of Incentive Mechanism Design for Federated Learning. *IEEE Transactions on Emerging Topics in Computing* 10, 2 (2022), 1035–1044.

[124] Bingxue Zhang, Guangguang Lu, Yuncheng Wu, Kunpeng Ren, and Feida Zhu. 2025. Zkfhed: A Verifiable and Scalable Blockchain-Enhanced Federated Learning System. *IEEE Transactions on Knowledge and Data Engineering* 37, 6 (2025), 3841–3854.

[125] Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, Qin Xie, Guiming Xie, and Xuejian Gong. 2025. HMI: hierarchical knowledge management for efficient multi-tenant inference in pretrained language models. *The VLDB Journal* 34, 4 (2025), 43.

[126] Peng Zhang, Gang Liu, Zhiyu Chen, Jianwei Guo, and Peng Liu. 2022. A study of a federated learning framework based on the interstellar file system and blockchain: Private Blockchain Federated Learning. In *2022 3rd International Conference on Computer Vision, Image and Deep Learning & International Conference on Computer Engineering and Applications (CVIDL & ICCEA)*. IEEE, 267–273.

[127] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. TinyLlama: An Open-Source Small Language Model. arXiv:2401.02385 [cs.CL]

[128] Zhebin Zhang, Dajie Dong, Yuhang Ma, Yilong Ying, Dawei Jiang, Ke Chen, Lidan Shou, and Gang Chen. 2021. Refiner: a reliable incentive-driven federated learning system powered by blockchain. *Proc. VLDB Endow.* 14, 12 (2021), 2659–2662.

[129] Zhuo Zhang, Yuanhang Yang, Yong Dai, Qifan Wang, Yue Yu, Lizhen Qu, and Zenglin Xu. 2023. FedPETuning: When federated learning meets the parameter-efficient tuning methods of pre-trained language models. In *Annual Meeting of the Association of Computational Linguistics 2023*. Association for Computational Linguistics (ACL), 9963–9977.

[130] Juncen Zhu, Jiannong Cao, Divya Saxena, Shan Jiang, and Houda Ferradi. 2023. Blockchain-empowered federated learning: Challenges, solutions, and future directions. *Comput. Surveys* 55, 11 (2023), 1–31.

[131] ZIRCONTech. 2025. The Convergence of AI and Web3: Building the Future of Decentralized Intelligence. https://zircon.tech/blog/the-convergence-of-ai-and-web3-building-the-future-of-decentralized-intelligence/